```
game.js  // ===== 文件：game.js =====
game.js:1  require('./src/main');
game.js
game.json  // ===== 文件：game.json =====
game.json:1  {
game.json:2    "deviceOrientation": "portrait",
game.json:3    "showStatusBar": false,
game.json:4    "networkTimeout": {
game.json:5      "request": 5000,
game.json:6      "connectSocket": 5000,
game.json:7      "uploadFile": 5000,
game.json:8      "downloadFile": 5000
game.json:9    }
game.json:10  }
game.json
project.config.j ...  // ===== 文件：project.config.json =====
project.config.j ...  {
project.config.j ...    "description": "Original gesture spell mini game MVP",
project.config.j ...    "appid": "wxb8fc97b860b32dd8",
project.config.j ...    "projectname": "gesture-spell-mvp",
project.config.j ...    "compileType": "game",
project.config.j ...    "libVersion": "3.7.12",
project.config.j ...    "setting": {
project.config.j ...      "urlCheck": false,
project.config.j ...      "es6": true,
project.config.j ...      "enhance": true,
project.config.j ...      "postcss": false,
project.config.j ...      "minified": false,
project.config.j ...      "uglifyFileName": false,
project.config.j ...      "compileWorklet": false,
project.config.j ...      "uploadWithSourceMap": true,
project.config.j ...      "packNpmManually": false,
project.config.j ...      "packNpmRelationList": [],
project.config.j ...      "minifyWXSS": true,
project.config.j ...      "minifyWXML": true,
project.config.j ...      "localPlugins": false,
project.config.j ...      "disableUseStrict": false,
project.config.j ...      "useCompilerPlugins": false,
project.config.j ...      "condition": false,
project.config.j ...      "swc": false,
project.config.j ...      "disableSWC": true,
project.config.j ...      "babelSetting": {
project.config.j ...        "ignore": [],
project.config.j ...        "disablePlugins": [],
project.config.j ...        "outputPath": ""
project.config.j ...      }
project.config.j ...    },
project.config.j ...    "condition": {},
project.config.j ...    "simulatorPluginLibVersion": {},
project.config.j ...    "packOptions": {
```

```
project.config.j ...         "ignore": [
project.config.j ...           {
project.config.j ...             "type": "folder",
project.config.j ...             "value": "scripts"
project.config.j ...           },
project.config.j ...           {
project.config.j ...             "type": "folder",
project.config.j ...             "value": "test"
project.config.j ...           },
project.config.j ...           {
project.config.j ...             "type": "file",
project.config.j ...             "value": "README.md"
project.config.j ...           },
project.config.j ...           {
project.config.j ...             "type": "file",
project.config.j ...             "value": "assets/images/castle-hall-bg.jpg"
project.config.j ...           }
project.config.j ...         ],
project.config.j ...         "include": []
project.config.j ...       },
project.config.j ...       "isGameTourist": false,
project.config.j ...       "editorSetting": {}
project.config.j ...     }
project.config.j ...
          src/main.js    // ===== 文件：src/main.js =====
        src/main.js:1    'use strict';
        src/main.js:2
        src/main.js:3    const { GameState } = require('./core/GameState');
        src/main.js:4    const InputManager = require('./input/InputManager');
        src/main.js:5    const Renderer = require('./render/Renderer');
        src/main.js:6    const SoundManager = require('./audio/SoundManager');
        src/main.js:7    const AssetManager = require('./assets/AssetManager');
        src/main.js:8
        src/main.js:9    function getSystemInfo(wxApi) {
       src/main.js:10      if (wxApi && wxApi.getSystemInfoSync) {
       src/main.js:11        return wxApi.getSystemInfoSync();
       src/main.js:12      }
       src/main.js:13      return {
       src/main.js:14        windowWidth: 375,
       src/main.js:15        windowHeight: 667,
       src/main.js:16        pixelRatio: 1
       src/main.js:17      };
       src/main.js:18    }
       src/main.js:19
       src/main.js:20    function createCanvas(wxApi) {
       src/main.js:21      if (wxApi && wxApi.createCanvas) {
       src/main.js:22        return wxApi.createCanvas();
       src/main.js:23      }
       src/main.js:24
       src/main.js:25      if (typeof document !== 'undefined') {
```

```
src/main.js:26     const canvas = document.createElement('canvas');
src/main.js:27     document.body.appendChild(canvas);
src/main.js:28     return canvas;
src/main.js:29   }
src/main.js:30
src/main.js:31   throw new Error('Canvas is only available in WeChat Mini Game or a browser preview.');
src/main.js:32 }
src/main.js:33
src/main.js:34 function ensureRoundRect(ctx) {
src/main.js:35   if (ctx.roundRect) {
src/main.js:36     return;
src/main.js:37   }
src/main.js:38
src/main.js:39   ctx.roundRect = function roundRect(x, y, width, height, radius) {
src/main.js:40     const r = Math.min(radius, width / 2, height / 2);
src/main.js:41     this.moveTo(x + r, y);
src/main.js:42     this.lineTo(x + width - r, y);
src/main.js:43     this.quadraticCurveTo(x + width, y, x + width, y + r);
src/main.js:44     this.lineTo(x + width, y + height - r);
src/main.js:45     this.quadraticCurveTo(x + width, y + height, x + width - r, y + height);
src/main.js:46     this.lineTo(x + r, y + height);
src/main.js:47     this.quadraticCurveTo(x, y + height, x, y + height - r);
src/main.js:48     this.lineTo(x, y + r);
src/main.js:49     this.quadraticCurveTo(x, y, x + r, y);
src/main.js:50   };
src/main.js:51 }
src/main.js:52
src/main.js:53 function createRaf(canvas) {
src/main.js:54   if (typeof requestAnimationFrame === 'function') {
src/main.js:55     return requestAnimationFrame;
src/main.js:56   }
src/main.js:57
src/main.js:58   if (canvas && canvas.requestAnimationFrame) {
src/main.js:59     return canvas.requestAnimationFrame.bind(canvas);
src/main.js:60   }
src/main.js:61
src/main.js:62   return function raf(fn) {
src/main.js:63     return setTimeout(function tick() {
src/main.js:64       fn(Date.now());
src/main.js:65     }, 1000 / 60);
src/main.js:66   };
src/main.js:67 }
src/main.js:68
src/main.js:69 function bootstrap() {
src/main.js:70   const wxApi = typeof wx !== 'undefined' ? wx : null;
src/main.js:71   const info = getSystemInfo(wxApi);
src/main.js:72   const canvas = createCanvas(wxApi);
src/main.js:73   const ratio = info.pixelRatio || 1;
src/main.js:74   const width = info.windowWidth || 375;
src/main.js:75   const height = info.windowHeight || 667;
```

```
src/main.js:76
src/main.js:77  canvas.width = Math.floor(width * ratio);
src/main.js:78  canvas.height = Math.floor(height * ratio);
src/main.js:79  canvas.style = canvas.style || {};
src/main.js:80  canvas.style.width = width + 'px';
src/main.js:81  canvas.style.height = height + 'px';
src/main.js:82
src/main.js:83  const ctx = canvas.getContext('2d');
src/main.js:84  ctx.scale(ratio, ratio);
src/main.js:85  ensureRoundRect(ctx);
src/main.js:86
src/main.js:87  const assets = new AssetManager(wxApi);
src/main.js:88  assets.loadImage('catWalk', 'assets/images/cat-walk-v2-sheet.png');
src/main.js:89  assets.loadImage('catCast', 'assets/images/cat-cast-v2-sheet.png');
src/main.js:90  assets.loadImage('catHurt', 'assets/images/cat-hurt-v2-sheet.png');
src/main.js:91  assets.loadImage('symbolUp', 'assets/images/symbol-up.png');
src/main.js:92  assets.loadImage('symbolDown', 'assets/images/symbol-down.png');
src/main.js:93  assets.loadImage('symbolLeft', 'assets/images/symbol-left.png');
src/main.js:94  assets.loadImage('symbolRight', 'assets/images/symbol-right.png');
src/main.js:95  assets.loadImage('symbolV', 'assets/images/symbol-v.png');
src/main.js:96  assets.loadImage('symbolL', 'assets/images/symbol-l.png');
src/main.js:97  assets.loadImage('symbolCircle', 'assets/images/symbol-circle.png');
src/main.js:98  assets.loadImage('symbolZ', 'assets/images/symbol-z.png');
src/main.js:99  assets.loadImage('symbolM', 'assets/images/symbol-m.png');
src/main.js:100 assets.loadImage('symbolS', 'assets/images/symbol-s.png');
src/main.js:101 assets.loadImage('castleCorridorLoop', 'assets/images/castle-corridor-loop.jpg');
src/main.js:102 assets.loadImage('oceanSpaceshipCorridorLoop', 'assets/images/ocean-spaceship-corridor-loop.jpg');
src/main.js:103 assets.loadImage('enemyJellyfish', 'assets/images/enemy-jellyfish-sheet.png');
src/main.js:104 assets.loadImage('enemyPufferfish', 'assets/images/enemy-pufferfish-sheet.png');
src/main.js:105 assets.loadImage('enemyShark', 'assets/images/enemy-shark-sheet.png');
src/main.js:106 assets.loadImage('bossMegalodon', 'assets/images/boss-megalodon-sheet.png');
src/main.js:107 assets.loadImage('penguinHotelCorridorLoop', 'assets/images/penguin-hotel-corridor-loop.jpg');
src/main.js:108 assets.loadImage('enemyPenguinBellhop', 'assets/images/enemy-penguin-bellhop-sheet.png');
src/main.js:109 assets.loadImage('enemyPenguinChef', 'assets/images/enemy-penguin-chef-sheet.png');
src/main.js:110 assets.loadImage('bossEmperorPenguin', 'assets/images/boss-emperor-penguin-sheet.png');
src/main.js:111 assets.loadImage('dinosaurParkCorridorLoop', 'assets/images/dinosaur-park-corridor-loop.jpg');
src/main.js:112 assets.loadImage('enemyPterosaur', 'assets/images/enemy-pterosaur-sheet.png');
src/main.js:113 assets.loadImage('enemyTriceratops', 'assets/images/enemy-triceratops-sheet.png');
src/main.js:114 assets.loadImage('enemyBrachiosaurus', 'assets/images/enemy-brachiosaurus-sheet.png');
src/main.js:115 assets.loadImage('bossTyrannosaurus', 'assets/images/boss-tyrannosaurus-sheet.png');
src/main.js:116
src/main.js:117 const sound = new SoundManager(wxApi);
src/main.js:118 const state = new GameState(width, height, sound);
src/main.js:119 state.healthNoticeVisible = true;
src/main.js:120 const input = new InputManager(wxApi, canvas);
src/main.js:121 const renderer = new Renderer(ctx, { width, height }, assets);
src/main.js:122
src/main.js:123 input.onGesture = function onGesture(points) {
src/main.js:124   state.handleGesture(points);
src/main.js:125 };
```

```
src/main.js:126  input.onTap = function onTap(point) {
src/main.js:127    if (state.healthNoticeVisible) {
src/main.js:128      state.healthNoticeVisible = false;
src/main.js:129      return;
src/main.js:130    }
src/main.js:131    state.handleTap(point);
src/main.js:132  };
src/main.js:133  input.bind();
src/main.js:134
src/main.js:135  let last = Date.now();
src/main.js:136  const raf = createRaf(canvas);
src/main.js:137
src/main.js:138  function loop() {
src/main.js:139    const now = Date.now();
src/main.js:140    const dt = Math.min(0.033, Math.max(0, (now - last) / 1000));
src/main.js:141    last = now;
src/main.js:142    state.update(dt);
src/main.js:143    renderer.render(state, input);
src/main.js:144    raf(loop);
src/main.js:145  }
src/main.js:146
src/main.js:147  loop();
src/main.js:148 }
src/main.js:149
src/main.js:150 bootstrap();
      src/main.js
src/core/Game ... // ===== 文件：src/core/GameState.js =====
src/core/Game ... 'use strict';
src/core/Game ...
src/core/Game ... const LevelDirector = require('../levels/LevelDirector');
src/core/Game ... const { THEME_IDS, createThemeOrder } = require('../levels/Themes');
src/core/Game ... const Enemy = require('../entities/Enemy');
src/core/Game ... const { recognize, SYMBOLS } = require('../input/GestureRecognizer');
src/core/Game ... const { DIFFICULTY_MODES, getDifficulty, findDifficultyAtPoint } = require('../ui/DifficultySelector');
src/core/Game ... const { isAudioToggleHit } = require('../ui/AudioToggle');
src/core/Game ... const { ITEM_TYPES, getItemSlots, findItemAtPoint } = require('../ui/ItemBar');
src/core/Game ...
src/core/Game ... const SCREENS = {
src/core/Game ...   TITLE: 'title',
src/core/Game ...   PLAYING: 'playing',
src/core/Game ...   LEVEL_TRANSITION: 'level-transition',
src/core/Game ...   WIN: 'win',
src/core/Game ...   LOSE: 'lose'
src/core/Game ... };
src/core/Game ...
src/core/Game ... const ANIMATION_DURATIONS = {
src/core/Game ...   cast: 0.38,
src/core/Game ...   impact: 0.52,
src/core/Game ...   vanish: 0.62,
src/core/Game ...   lightning: 0.3,
```

```
src/core/Game ...     comboChain: 0.5,
src/core/Game ...     heartLoss: 0.62,
src/core/Game ...     potionToHeart: 0.5,
src/core/Game ...     itemEarn: 0.72
src/core/Game ...   };
src/core/Game ...
src/core/Game ...   const HERO_ANIMATION_DURATIONS = {
src/core/Game ...     cast: 0.8,
src/core/Game ...     hurt: 0.9
src/core/Game ...   };
src/core/Game ...
src/core/Game ...   const MAX_LIVES = 5;
src/core/Game ...
src/core/Game ...   function getComboMultiplier(combo) {
src/core/Game ...     if (combo >= 100) {
src/core/Game ...       return 4;
src/core/Game ...     }
src/core/Game ...     if (combo >= 50) {
src/core/Game ...       return 3;
src/core/Game ...     }
src/core/Game ...     if (combo >= 20) {
src/core/Game ...       return 2.5;
src/core/Game ...     }
src/core/Game ...     if (combo >= 10) {
src/core/Game ...       return 2;
src/core/Game ...     }
src/core/Game ...     if (combo >= 5) {
src/core/Game ...       return 1.5;
src/core/Game ...     }
src/core/Game ...     if (combo >= 3) {
src/core/Game ...       return 1.2;
src/core/Game ...     }
src/core/Game ...     return 1;
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   class GameState {
src/core/Game ...     constructor(width, height, sound, random) {
src/core/Game ...       this.width = width;
src/core/Game ...       this.height = height;
src/core/Game ...       this.sound = sound;
src/core/Game ...       this.random = random || Math.random;
src/core/Game ...       this.themeOrder = createThemeOrder(this.random);
src/core/Game ...       this.hero = this.createHero(width, height);
src/core/Game ...       this.director = new LevelDirector(width, height);
src/core/Game ...       this.difficulty = null;
src/core/Game ...       this.timeScale = 1;
src/core/Game ...       this.soundEnabled = true;
src/core/Game ...       if (this.sound && this.sound.setSoundEnabled) {
src/core/Game ...         this.sound.setSoundEnabled(true);
src/core/Game ...       }
```

```
src/core/Game ...     this.screen = SCREENS.TITLE;
src/core/Game ...     this.resetRun();
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   createHero(width, height) {
src/core/Game ...     return {
src/core/Game ...       x: width * 0.5,
src/core/Game ...       y: height * 0.74,
src/core/Game ...       radius: Math.max(28, Math.min(width, height) * 0.07)
src/core/Game ...     };
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   resize(width, height) {
src/core/Game ...     this.width = width;
src/core/Game ...     this.height = height;
src/core/Game ...     this.hero = this.createHero(width, height);
src/core/Game ...     this.director.width = width;
src/core/Game ...     this.director.height = height;
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   resetRun() {
src/core/Game ...     this.score = 0;
src/core/Game ...     this.maxLives = MAX_LIVES;
src/core/Game ...     this.lives = this.maxLives;
src/core/Game ...     this.level = 1;
src/core/Game ...     this.totalLevels = this.director.levelCount;
src/core/Game ...     this.elapsed = 0;
src/core/Game ...     this.transitionRemaining = 0;
src/core/Game ...     this.combo = 0;
src/core/Game ...     this.items = {
src/core/Game ...       comboChain: 0,
src/core/Game ...       healthPotion: 0
src/core/Game ...     };
src/core/Game ...     this.enemies = [];
src/core/Game ...     this.feedback = null;
src/core/Game ...     this.effects = [];
src/core/Game ...     this.heroAnimation = {
src/core/Game ...       cast: 0,
src/core/Game ...       castAge: 0,
src/core/Game ...       hurt: 0,
src/core/Game ...       hurtAge: 0
src/core/Game ...     };
src/core/Game ...     this.director.reset(this.width, this.height, this.level, this.getThemeId(this.level));
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   getThemeId(level) {
src/core/Game ...     return this.themeOrder[(level || 1) - 1] || THEME_IDS.CASTLE;
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   createNewThemeOrder() {
```

```
src/core/Game ...     this.themeOrder = createThemeOrder(this.random);
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   start(difficultyMode) {
src/core/Game ...     this.selectDifficulty(difficultyMode);
src/core/Game ...     this.resetRun();
src/core/Game ...     this.screen = SCREENS.PLAYING;
src/core/Game ...     if (this.sound) {
src/core/Game ...       this.sound.play('start');
src/core/Game ...       if (this.sound.playMusic) {
src/core/Game ...         this.sound.playMusic(this.getThemeId(this.level));
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   update(dt) {
src/core/Game ...     this.updateAnimations(dt);
src/core/Game ...
src/core/Game ...     if (this.screen === SCREENS.LEVEL_TRANSITION) {
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (this.screen !== SCREENS.PLAYING) {
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     const playDt = dt * this.timeScale;
src/core/Game ...     this.elapsed += playDt;
src/core/Game ...     this.director.update(this.elapsed, this.enemies);
src/core/Game ...
src/core/Game ...     const livesBeforeDamage = this.lives;
src/core/Game ...     let damageTaken = 0;
src/core/Game ...     for (let i = 0; i < this.enemies.length; i += 1) {
src/core/Game ...       const enemy = this.enemies[i];
src/core/Game ...       enemy.update(dt, this.hero, this.timeScale);
src/core/Game ...       if (enemy.reachedHero) {
src/core/Game ...         this.lives -= 1;
src/core/Game ...         this.combo = 0;
src/core/Game ...         damageTaken += 1;
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (damageTaken > 0) {
src/core/Game ...       this.feedback = { text: '爱心 -' + damageTaken, age: 0, type: 'hurt' };
src/core/Game ...       this.triggerHeroImpact();
src/core/Game ...       this.triggerHeartLoss(livesBeforeDamage, damageTaken);
src/core/Game ...       if (this.sound) {
src/core/Game ...         this.sound.play('hurt');
src/core/Game ...         if (this.sound.vibrateDamage) {
src/core/Game ...           this.sound.vibrateDamage();
```

```
src/core/Game ...         }
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     this.enemies = this.enemies.filter((enemy) => !enemy.dead);
src/core/Game ...
src/core/Game ...     if (this.lives <= 0) {
src/core/Game ...       this.screen = SCREENS.LOSE;
src/core/Game ...       if (this.sound) {
src/core/Game ...         if (this.sound.stopMusic) {
src/core/Game ...           this.sound.stopMusic();
src/core/Game ...         }
src/core/Game ...         this.sound.play('lose');
src/core/Game ...       }
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (this.director.isComplete(this.enemies)) {
src/core/Game ...       if (this.level < this.totalLevels) {
src/core/Game ...         this.beginLevelTransition();
src/core/Game ...       } else {
src/core/Game ...         this.screen = SCREENS.WIN;
src/core/Game ...         if (this.sound) {
src/core/Game ...           if (this.sound.stopMusic) {
src/core/Game ...             this.sound.stopMusic();
src/core/Game ...           }
src/core/Game ...           this.sound.play('win');
src/core/Game ...         }
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   beginLevelTransition() {
src/core/Game ...     this.screen = SCREENS.LEVEL_TRANSITION;
src/core/Game ...     this.transitionRemaining = 0;
src/core/Game ...     this.combo = 0;
src/core/Game ...     this.feedback = null;
src/core/Game ...     this.enemies = [];
src/core/Game ...     if (this.sound && this.sound.playMusic) {
src/core/Game ...       this.sound.playMusic(this.getThemeId(this.level + 1));
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   selectDifficulty(difficultyMode) {
src/core/Game ...     const difficulty = getDifficulty(difficultyMode || DIFFICULTY_MODES.NORMAL);
src/core/Game ...     this.difficulty = difficulty.id;
src/core/Game ...     this.timeScale = difficulty.timeScale;
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   startNextLevel(difficultyMode) {
```

```
src/core/Game ...     this.selectDifficulty(difficultyMode);
src/core/Game ...     this.level += 1;
src/core/Game ...     this.elapsed = 0;
src/core/Game ...     this.combo = 0;
src/core/Game ...     this.feedback = null;
src/core/Game ...     this.enemies = [];
src/core/Game ...     this.effects = [];
src/core/Game ...     this.director.startLevel(this.level, this.getThemeId(this.level));
src/core/Game ...     this.screen = SCREENS.PLAYING;
src/core/Game ...     if (this.sound) {
src/core/Game ...       this.sound.play('start');
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   updateAnimations(dt) {
src/core/Game ...     if (this.heroAnimation.cast > 0) {
src/core/Game ...       this.heroAnimation.castAge += dt;
src/core/Game ...     }
src/core/Game ...     if (this.heroAnimation.hurt > 0) {
src/core/Game ...       this.heroAnimation.hurtAge += dt;
src/core/Game ...     }
src/core/Game ...     this.heroAnimation.cast = Math.max(0, this.heroAnimation.cast - dt);
src/core/Game ...     this.heroAnimation.hurt = Math.max(0, this.heroAnimation.hurt - dt);
src/core/Game ...     for (let i = 0; i < this.effects.length; i += 1) {
src/core/Game ...       this.effects[i].age += dt;
src/core/Game ...     }
src/core/Game ...     this.effects = this.effects.filter((effect) => effect.age < effect.duration);
src/core/Game ...
src/core/Game ...     if (this.feedback) {
src/core/Game ...       this.feedback.age += dt;
src/core/Game ...       if (this.feedback.age > 0.7) {
src/core/Game ...         this.feedback = null;
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   addEffect(type, properties) {
src/core/Game ...     this.effects.push(Object.assign({
src/core/Game ...       type,
src/core/Game ...       age: 0,
src/core/Game ...       duration: ANIMATION_DURATIONS[type]
src/core/Game ...     }, properties));
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerCast(symbol) {
src/core/Game ...     this.heroAnimation.cast = HERO_ANIMATION_DURATIONS.cast;
src/core/Game ...     this.heroAnimation.castAge = 0;
src/core/Game ...     this.addEffect('cast', {
src/core/Game ...       x: this.hero.x + this.hero.radius * 0.8,
src/core/Game ...       y: this.hero.y - this.hero.radius * 0.85,
```

```
src/core/Game ...       radius: this.hero.radius,
src/core/Game ...       symbol
src/core/Game ...     });
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerHeroImpact() {
src/core/Game ...     this.heroAnimation.hurt = HERO_ANIMATION_DURATIONS.hurt;
src/core/Game ...     this.heroAnimation.hurtAge = 0;
src/core/Game ...     this.addEffect('impact', {
src/core/Game ...       x: this.hero.x,
src/core/Game ...       y: this.hero.y - this.hero.radius * 0.2,
src/core/Game ...       radius: this.hero.radius
src/core/Game ...     });
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerHeartLoss(previousLives, damageTaken) {
src/core/Game ...     const lostHearts = Math.min(Math.max(previousLives, 0), damageTaken);
src/core/Game ...     for (let i = 0; i < lostHearts; i += 1) {
src/core/Game ...       this.addEffect('heartLoss', {
src/core/Game ...         heartIndex: previousLives - 1 - i,
src/core/Game ...         burstIndex: i,
src/core/Game ...         burstCount: lostHearts
src/core/Game ...       });
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerPotionToHeart(origin, heartIndex) {
src/core/Game ...     this.addEffect('potionToHeart', {
src/core/Game ...       x: origin.x,
src/core/Game ...       y: origin.y,
src/core/Game ...       radius: origin.radius,
src/core/Game ...       heartIndex
src/core/Game ...     });
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerItemEarn(itemType) {
src/core/Game ...     const slot = getItemSlots(this.width, this.height, this.items).find((entry) => entry.type === itemType);
src/core/Game ...     if (!slot) {
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     this.addEffect('itemEarn', {
src/core/Game ...       itemType,
src/core/Game ...       toX: slot.x + slot.width * 0.5,
src/core/Game ...       toY: slot.y + slot.height * 0.5
src/core/Game ...     });
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerVanish(enemy) {
src/core/Game ...     this.addEffect('vanish', {
```

```
src/core/Game ...       x: enemy.x,
src/core/Game ...       y: enemy.y,
src/core/Game ...       radius: enemy.radius,
src/core/Game ...       kind: enemy.kind
src/core/Game ...     });
src/core/Game ...     this.addEffect('lightning', {
src/core/Game ...       fromX: this.hero.x + this.hero.radius * 0.72,
src/core/Game ...       fromY: this.hero.y - this.hero.radius * 0.92,
src/core/Game ...       toX: enemy.x,
src/core/Game ...       toY: enemy.y,
src/core/Game ...       radius: enemy.radius,
src/core/Game ...       kind: enemy.kind
src/core/Game ...     });
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   triggerComboChain(targets) {
src/core/Game ...     if (!targets.length) {
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     this.addEffect('comboChain', {
src/core/Game ...       originX: this.hero.x + this.hero.radius * 0.72,
src/core/Game ...       originY: this.hero.y - this.hero.radius * 0.9,
src/core/Game ...       targets: targets.map((enemy) => ({
src/core/Game ...         x: enemy.x,
src/core/Game ...         y: enemy.y,
src/core/Game ...         radius: enemy.radius,
src/core/Game ...         kind: enemy.kind,
src/core/Game ...         dead: enemy.dead
src/core/Game ...       }))
src/core/Game ...     });
src/core/Game ...
src/core/Game ...     for (let i = 0; i < targets.length; i += 1) {
src/core/Game ...       const enemy = targets[i];
src/core/Game ...       const hit = enemy.takeComboHit();
src/core/Game ...       if (hit && enemy.dead) {
src/core/Game ...         this.triggerVanish(enemy);
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...     this.enemies = this.enemies.filter((enemy) => !enemy.dead);
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   toggleSound() {
src/core/Game ...     this.soundEnabled = !this.soundEnabled;
src/core/Game ...     if (this.sound && this.sound.setSoundEnabled) {
src/core/Game ...       this.sound.setSoundEnabled(this.soundEnabled);
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   handleTap(point) {
```

```
src/core/Game ...     if (isAudioToggleHit(this.width, point)) {
src/core/Game ...       this.toggleSound();
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (this.screen === SCREENS.TITLE) {
src/core/Game ...       const selectedDifficulty = findDifficultyAtPoint(this.width, this.height, point);
src/core/Game ...       if (selectedDifficulty) {
src/core/Game ...         this.start(selectedDifficulty);
src/core/Game ...       }
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (this.screen === SCREENS.LEVEL_TRANSITION) {
src/core/Game ...       const selectedDifficulty = findDifficultyAtPoint(this.width, this.height, point);
src/core/Game ...       if (selectedDifficulty) {
src/core/Game ...         this.startNextLevel(selectedDifficulty);
src/core/Game ...       }
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (this.screen === SCREENS.PLAYING) {
src/core/Game ...       const itemType = findItemAtPoint(this.width, this.height, this.items, point);
src/core/Game ...       if (itemType) {
src/core/Game ...         this.useItem(itemType);
src/core/Game ...       }
src/core/Game ...       return;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (this.screen === SCREENS.WIN || this.screen === SCREENS.LOSE) {
src/core/Game ...       this.difficulty = null;
src/core/Game ...       this.timeScale = 1;
src/core/Game ...       this.createNewThemeOrder();
src/core/Game ...       this.resetRun();
src/core/Game ...       this.screen = SCREENS.TITLE;
src/core/Game ...     }
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   useItem(itemType) {
src/core/Game ...     if (!this.items[itemType] || this.items[itemType] <= 0) {
src/core/Game ...       return false;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (itemType === ITEM_TYPES.COMBO_CHAIN) {
src/core/Game ...       const targets = this.enemies.filter((enemy) => !enemy.dead);
src/core/Game ...       if (targets.length === 0) {
src/core/Game ...         this.feedback = { text: '没有可攻击目标', age: 0, type: 'item' };
src/core/Game ...         return false;
src/core/Game ...       }
src/core/Game ...
```

```
src/core/Game ...       this.items.comboChain -= 1;
src/core/Game ...       this.triggerComboChain(targets);
src/core/Game ...       this.feedback = { text: '紫色闪电释放', age: 0, type: 'hit' };
src/core/Game ...       return true;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     if (itemType === ITEM_TYPES.HEALTH_POTION) {
src/core/Game ...       if (this.lives >= this.maxLives) {
src/core/Game ...         this.feedback = { text: '爱心已满', age: 0, type: 'item' };
src/core/Game ...         return false;
src/core/Game ...       }
src/core/Game ...
src/core/Game ...       const slots = getItemSlots(this.width, this.height, this.items);
src/core/Game ...       const slot = slots.find((entry) => entry.type === ITEM_TYPES.HEALTH_POTION);
src/core/Game ...       this.items.healthPotion -= 1;
src/core/Game ...       const recoveredHeartIndex = this.lives;
src/core/Game ...       this.lives += 1;
src/core/Game ...       this.triggerPotionToHeart({
src/core/Game ...         x: slot.x + slot.width * 0.5,
src/core/Game ...         y: slot.y + slot.height * 0.5,
src/core/Game ...         radius: 22
src/core/Game ...       }, recoveredHeartIndex);
src/core/Game ...       this.feedback = { text: '爱心 +1', age: 0, type: 'hit' };
src/core/Game ...       return true;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     return false;
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   handleGesture(points) {
src/core/Game ...     if (this.screen !== SCREENS.PLAYING) {
src/core/Game ...       return SYMBOLS.UNKNOWN;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     const symbol = recognize(points);
src/core/Game ...     this.triggerCast(symbol);
src/core/Game ...     const targets = this.findMatchingEnemies(symbol);
src/core/Game ...     if (targets.length === 0) {
src/core/Game ...       this.combo = 0;
src/core/Game ...       this.feedback = { text: 'Miss', age: 0, type: 'miss' };
src/core/Game ...       if (this.sound) {
src/core/Game ...         this.sound.play('miss');
src/core/Game ...       }
src/core/Game ...       return symbol;
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     let vanished = 0;
src/core/Game ...     let partialHits = 0;
src/core/Game ...     let vanishedScore = 0;
src/core/Game ...     for (let i = 0; i < targets.length; i += 1) {
```

```
src/core/Game ...       const target = targets[i];
src/core/Game ...       target.applySymbol(symbol);
src/core/Game ...       if (target.symbols.length === 0) {
src/core/Game ...         vanished += 1;
src/core/Game ...         vanishedScore += target.score;
src/core/Game ...         this.triggerVanish(target);
src/core/Game ...       } else {
src/core/Game ...         partialHits += 1;
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     this.combo += 1;
src/core/Game ...     const multiplier = getComboMultiplier(this.combo);
src/core/Game ...     const earned = Math.round(vanishedScore * multiplier) + partialHits * 30;
src/core/Game ...     this.score += earned;
src/core/Game ...     const scoreFeedback = vanished > 0
src/core/Game ...       ? '消除 +' + earned
src/core/Game ...       : (partialHits > 0 ? '命中 +' + earned : '');
src/core/Game ...     let feedbackText = scoreFeedback;
src/core/Game ...     const earnedItems = [];
src/core/Game ...     if (this.combo % 10 === 0) {
src/core/Game ...       this.items.comboChain += 1;
src/core/Game ...       earnedItems.push(ITEM_TYPES.COMBO_CHAIN);
src/core/Game ...       feedbackText += '  获得紫色闪电';
src/core/Game ...     }
src/core/Game ...     if (this.combo % 15 === 0) {
src/core/Game ...       this.items.healthPotion += 1;
src/core/Game ...       earnedItems.push(ITEM_TYPES.HEALTH_POTION);
src/core/Game ...       feedbackText += '  获得血瓶';
src/core/Game ...     }
src/core/Game ...     for (let i = 0; i < earnedItems.length; i += 1) {
src/core/Game ...       this.triggerItemEarn(earnedItems[i]);
src/core/Game ...     }
src/core/Game ...     this.feedback = {
src/core/Game ...       text: feedbackText,
src/core/Game ...       age: 0,
src/core/Game ...       type: 'hit',
src/core/Game ...       combo: this.combo,
src/core/Game ...       multiplier: vanished > 0 ? multiplier : null
src/core/Game ...     };
src/core/Game ...     this.enemies = this.enemies.filter((enemy) => !enemy.dead);
src/core/Game ...     if (this.sound) {
src/core/Game ...       this.sound.play(vanished > 0 ? 'vanish' : 'hit');
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     return symbol;
src/core/Game ...   }
src/core/Game ...
src/core/Game ...   findMatchingEnemies(symbol) {
src/core/Game ...     if (symbol === SYMBOLS.UNKNOWN) {
```

```
src/core/Game ...       return [];
src/core/Game ...     }
src/core/Game ...
src/core/Game ...     const matches = [];
src/core/Game ...     for (let i = 0; i < this.enemies.length; i += 1) {
src/core/Game ...       const enemy = this.enemies[i];
src/core/Game ...       if (!enemy.dead && enemy.matches(symbol)) {
src/core/Game ...         matches.push(enemy);
src/core/Game ...       }
src/core/Game ...     }
src/core/Game ...     return matches;
src/core/Game ...   }
src/core/Game ... }
src/core/Game ...
src/core/Game ... module.exports = {
src/core/Game ...   GameState,
src/core/Game ...   SCREENS,
src/core/Game ...   getComboMultiplier
src/core/Game ... };
src/core/Game ...
src/input/Input ... // ===== 文件：src/input/InputManager.js =====
src/input/Input ... 'use strict';
src/input/Input ...
src/input/Input ... class InputManager {
src/input/Input ...   constructor(wxApi, canvas) {
src/input/Input ...     this.wx = wxApi;
src/input/Input ...     this.canvas = canvas;
src/input/Input ...     this.currentPath = [];
src/input/Input ...     this.lastCompletedPath = null;
src/input/Input ...     this.onGesture = null;
src/input/Input ...     this.onTap = null;
src/input/Input ...     this.isDrawing = false;
src/input/Input ...
src/input/Input ...     this.handleStart = this.handleStart.bind(this);
src/input/Input ...     this.handleMove = this.handleMove.bind(this);
src/input/Input ...     this.handleEnd = this.handleEnd.bind(this);
src/input/Input ...   }
src/input/Input ...
src/input/Input ...   bind() {
src/input/Input ...     if (!this.wx) {
src/input/Input ...       return;
src/input/Input ...     }
src/input/Input ...
src/input/Input ...     this.wx.onTouchStart(this.handleStart);
src/input/Input ...     this.wx.onTouchMove(this.handleMove);
src/input/Input ...     this.wx.onTouchEnd(this.handleEnd);
src/input/Input ...     if (this.wx.onTouchCancel) {
src/input/Input ...       this.wx.onTouchCancel(this.handleEnd);
src/input/Input ...     }
src/input/Input ...   }
```

```
src/input/Input ...
src/input/Input ...  extractPoint(event) {
src/input/Input ...    const touch = event && event.touches && event.touches[0];
src/input/Input ...    const changed = event && event.changedTouches && event.changedTouches[0];
src/input/Input ...    const source = touch || changed;
src/input/Input ...    if (!source) {
src/input/Input ...      return null;
src/input/Input ...    }
src/input/Input ...
src/input/Input ...    return {
src/input/Input ...      x: source.clientX,
src/input/Input ...      y: source.clientY,
src/input/Input ...      t: Date.now()
src/input/Input ...    };
src/input/Input ...  }
src/input/Input ...
src/input/Input ...  handleStart(event) {
src/input/Input ...    const point = this.extractPoint(event);
src/input/Input ...    if (!point) {
src/input/Input ...      return;
src/input/Input ...    }
src/input/Input ...
src/input/Input ...    this.isDrawing = true;
src/input/Input ...    this.currentPath = [point];
src/input/Input ...    this.lastCompletedPath = null;
src/input/Input ...  }
src/input/Input ...
src/input/Input ...  handleMove(event) {
src/input/Input ...    if (!this.isDrawing) {
src/input/Input ...      return;
src/input/Input ...    }
src/input/Input ...
src/input/Input ...    const point = this.extractPoint(event);
src/input/Input ...    if (point) {
src/input/Input ...      this.currentPath.push(point);
src/input/Input ...    }
src/input/Input ...  }
src/input/Input ...
src/input/Input ...  handleEnd(event) {
src/input/Input ...    if (!this.isDrawing) {
src/input/Input ...      return;
src/input/Input ...    }
src/input/Input ...
src/input/Input ...    const point = this.extractPoint(event);
src/input/Input ...    if (point) {
src/input/Input ...      this.currentPath.push(point);
src/input/Input ...    }
src/input/Input ...
src/input/Input ...    const completed = this.currentPath.slice();
src/input/Input ...    this.lastCompletedPath = completed;
```

```
src/input/Input ...     this.currentPath = [];
src/input/Input ...     this.isDrawing = false;
src/input/Input ...
src/input/Input ...     if (completed.length <= 3) {
src/input/Input ...       if (this.onTap) {
src/input/Input ...         this.onTap(completed[0] || point);
src/input/Input ...       }
src/input/Input ...       return;
src/input/Input ...     }
src/input/Input ...
src/input/Input ...     if (this.onGesture) {
src/input/Input ...       this.onGesture(completed);
src/input/Input ...     }
src/input/Input ...   }
src/input/Input ... }
src/input/Input ...
src/input/Input ... module.exports = InputManager;
src/input/Input ...
src/input/Gestu ... // ===== 文件：src/input/GestureRecognizer.js =====
src/input/Gestu ... 'use strict';
src/input/Gestu ...
src/input/Gestu ... const SYMBOLS = {
src/input/Gestu ...   UP: 'up',
src/input/Gestu ...   DOWN: 'down',
src/input/Gestu ...   LEFT: 'left',
src/input/Gestu ...   RIGHT: 'right',
src/input/Gestu ...   V: 'v',
src/input/Gestu ...   L: 'l',
src/input/Gestu ...   CIRCLE: 'circle',
src/input/Gestu ...   Z: 'z',
src/input/Gestu ...   M: 'm',
src/input/Gestu ...   S: 's',
src/input/Gestu ...   UNKNOWN: 'unknown'
src/input/Gestu ... };
src/input/Gestu ...
src/input/Gestu ... const LABELS = {
src/input/Gestu ...   [SYMBOLS.UP]: '↑',
src/input/Gestu ...   [SYMBOLS.DOWN]: '↓',
src/input/Gestu ...   [SYMBOLS.LEFT]: '←',
src/input/Gestu ...   [SYMBOLS.RIGHT]: '→',
src/input/Gestu ...   [SYMBOLS.V]: 'V',
src/input/Gestu ...   [SYMBOLS.L]: 'L',
src/input/Gestu ...   [SYMBOLS.CIRCLE]: '○',
src/input/Gestu ...   [SYMBOLS.Z]: 'Z',
src/input/Gestu ...   [SYMBOLS.M]: 'M',
src/input/Gestu ...   [SYMBOLS.S]: 'S',
src/input/Gestu ...   [SYMBOLS.UNKNOWN]: '?'
src/input/Gestu ... };
src/input/Gestu ...
src/input/Gestu ... function getBounds(points) {
```

```
src/input/Gestu ...   let minX = Infinity;
src/input/Gestu ...   let minY = Infinity;
src/input/Gestu ...   let maxX = -Infinity;
src/input/Gestu ...   let maxY = -Infinity;
src/input/Gestu ...
src/input/Gestu ...   for (let i = 0; i < points.length; i += 1) {
src/input/Gestu ...     const p = points[i];
src/input/Gestu ...     minX = Math.min(minX, p.x);
src/input/Gestu ...     minY = Math.min(minY, p.y);
src/input/Gestu ...     maxX = Math.max(maxX, p.x);
src/input/Gestu ...     maxY = Math.max(maxY, p.y);
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return {
src/input/Gestu ...     minX,
src/input/Gestu ...     minY,
src/input/Gestu ...     maxX,
src/input/Gestu ...     maxY,
src/input/Gestu ...     width: maxX - minX,
src/input/Gestu ...     height: maxY - minY
src/input/Gestu ...   };
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function pathLength(points) {
src/input/Gestu ...   let total = 0;
src/input/Gestu ...   for (let i = 1; i < points.length; i += 1) {
src/input/Gestu ...     const dx = points[i].x - points[i - 1].x;
src/input/Gestu ...     const dy = points[i].y - points[i - 1].y;
src/input/Gestu ...     total += Math.sqrt(dx * dx + dy * dy);
src/input/Gestu ...   }
src/input/Gestu ...   return total;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function simplify(points) {
src/input/Gestu ...   if (!points || points.length < 2) {
src/input/Gestu ...     return [];
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const out = [points[0]];
src/input/Gestu ...   for (let i = 1; i < points.length; i += 1) {
src/input/Gestu ...     const prev = out[out.length - 1];
src/input/Gestu ...     const next = points[i];
src/input/Gestu ...     const dx = next.x - prev.x;
src/input/Gestu ...     const dy = next.y - prev.y;
src/input/Gestu ...     if (Math.sqrt(dx * dx + dy * dy) >= 4) {
src/input/Gestu ...       out.push(next);
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (out[out.length - 1] !== points[points.length - 1]) {
```

```
src/input/Gestu ...     out.push(points[points.length - 1]);
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return out;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function recognizeLine(points, bounds) {
src/input/Gestu ...   const first = points[0];
src/input/Gestu ...   const last = points[points.length - 1];
src/input/Gestu ...   const dx = last.x - first.x;
src/input/Gestu ...   const dy = last.y - first.y;
src/input/Gestu ...   const absX = Math.abs(dx);
src/input/Gestu ...   const absY = Math.abs(dy);
src/input/Gestu ...
src/input/Gestu ...   if (Math.max(absX, absY) < 22) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (absY > absX * 1.12 && bounds.height > 20) {
src/input/Gestu ...     return dy < 0 ? SYMBOLS.UP : SYMBOLS.DOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (absX > absY * 1.12 && bounds.width > 20) {
src/input/Gestu ...     return dx < 0 ? SYMBOLS.LEFT : SYMBOLS.RIGHT;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return SYMBOLS.UNKNOWN;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function findCornerIndex(points, wantLowY) {
src/input/Gestu ...   let bestIndex = -1;
src/input/Gestu ...   let bestScore = wantLowY ? Infinity : -Infinity;
src/input/Gestu ...
src/input/Gestu ...   for (let i = 1; i < points.length - 1; i += 1) {
src/input/Gestu ...     const p = points[i];
src/input/Gestu ...     if ((wantLowY && p.y < bestScore) || (!wantLowY && p.y > bestScore)) {
src/input/Gestu ...       bestScore = p.y;
src/input/Gestu ...       bestIndex = i;
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return bestIndex;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function hasLooseCorner(firstArm, secondArm, spread, bounds) {
src/input/Gestu ...   const longArm = Math.max(firstArm, secondArm);
src/input/Gestu ...   const shortArm = Math.min(firstArm, secondArm);
src/input/Gestu ...   return (
src/input/Gestu ...     longArm > bounds.height * 0.42 &&
src/input/Gestu ...     shortArm > Math.max(5, bounds.height * 0.08) &&
```

```
src/input/Gestu ...     spread > Math.max(6, bounds.width * 0.18)
src/input/Gestu ...   );
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function recognizeVShape(points, bounds) {
src/input/Gestu ...   if (points.length < 3 || bounds.width < 8 || bounds.height < 14) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const first = points[0];
src/input/Gestu ...   const last = points[points.length - 1];
src/input/Gestu ...   const bottomIndex = findCornerIndex(points, false);
src/input/Gestu ...   const bottom = points[bottomIndex];
src/input/Gestu ...
src/input/Gestu ...   if (bottom && bottomIndex > 0 && bottomIndex < points.length - 1) {
src/input/Gestu ...     const leftDrop = bottom.y - first.y;
src/input/Gestu ...     const rightRise = bottom.y - last.y;
src/input/Gestu ...     const spread = Math.abs(last.x - first.x);
src/input/Gestu ...     // Young players often finish one arm early or draw a very narrow corner.
src/input/Gestu ...     if (hasLooseCorner(leftDrop, rightRise, spread, bounds)) {
src/input/Gestu ...       return SYMBOLS.V;
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return SYMBOLS.UNKNOWN;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function pointDistance(a, b) {
src/input/Gestu ...   const dx = a.x - b.x;
src/input/Gestu ...   const dy = a.y - b.y;
src/input/Gestu ...   return Math.sqrt(dx * dx + dy * dy);
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function angularTravel(points, bounds) {
src/input/Gestu ...   const center = {
src/input/Gestu ...     x: bounds.minX + bounds.width / 2,
src/input/Gestu ...     y: bounds.minY + bounds.height / 2
src/input/Gestu ...   };
src/input/Gestu ...   let previous = Math.atan2(points[0].y - center.y, points[0].x - center.x);
src/input/Gestu ...   let signed = 0;
src/input/Gestu ...   let total = 0;
src/input/Gestu ...
src/input/Gestu ...   for (let i = 1; i < points.length; i += 1) {
src/input/Gestu ...     const next = Math.atan2(points[i].y - center.y, points[i].x - center.x);
src/input/Gestu ...     const delta = Math.atan2(Math.sin(next - previous), Math.cos(next - previous));
src/input/Gestu ...     signed += delta;
src/input/Gestu ...     total += Math.abs(delta);
src/input/Gestu ...     previous = next;
src/input/Gestu ...   }
src/input/Gestu ...
```

```
src/input/Gestu ...     return { signed, total };
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   function recognizeCircle(points, bounds) {
src/input/Gestu ...     if (points.length < 5 || bounds.width < 28 || bounds.height < 28) {
src/input/Gestu ...       return SYMBOLS.UNKNOWN;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     const ratio = bounds.width / bounds.height;
src/input/Gestu ...     const maxSide = Math.max(bounds.width, bounds.height);
src/input/Gestu ...     const perimeter = 2 * (bounds.width + bounds.height);
src/input/Gestu ...     if (ratio < 0.5 || ratio > 1.9) {
src/input/Gestu ...       return SYMBOLS.UNKNOWN;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     // Let players leave a visible gap when drawing a quick ring gesture.
src/input/Gestu ...     if (pointDistance(points[0], points[points.length - 1]) > maxSide * 0.84) {
src/input/Gestu ...       return SYMBOLS.UNKNOWN;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     if (pathLength(points) < perimeter * 0.48) {
src/input/Gestu ...       return SYMBOLS.UNKNOWN;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     const turn = angularTravel(points, bounds);
src/input/Gestu ...     if (Math.abs(turn.signed) < Math.PI * 1.16 || Math.abs(turn.signed) < turn.total * 0.58) {
src/input/Gestu ...       return SYMBOLS.UNKNOWN;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     return SYMBOLS.CIRCLE;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   function followsZDirection(points, bounds) {
src/input/Gestu ...     const first = points[0];
src/input/Gestu ...     const last = points[points.length - 1];
src/input/Gestu ...     if (
src/input/Gestu ...       first.x > bounds.minX + bounds.width * 0.45 ||
src/input/Gestu ...       first.y > bounds.minY + bounds.height * 0.45 ||
src/input/Gestu ...       last.x < bounds.maxX - bounds.width * 0.45 ||
src/input/Gestu ...       last.y < bounds.maxY - bounds.height * 0.45
src/input/Gestu ...     ) {
src/input/Gestu ...       return false;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     for (let topIndex = 1; topIndex < points.length - 2; topIndex += 1) {
src/input/Gestu ...       const topRight = points[topIndex];
src/input/Gestu ...       if (
src/input/Gestu ...         topRight.x - first.x < bounds.width * 0.4 ||
src/input/Gestu ...         Math.abs(topRight.y - first.y) > bounds.height * 0.4 ||
src/input/Gestu ...         topRight.y > bounds.minY + bounds.height * 0.43
```

```
src/input/Gestu ...     ) {
src/input/Gestu ...       continue;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     for (let bottomIndex = topIndex + 1; bottomIndex < points.length - 1; bottomIndex += 1) {
src/input/Gestu ...       const bottomLeft = points[bottomIndex];
src/input/Gestu ...       if (
src/input/Gestu ...         topRight.x - bottomLeft.x >= bounds.width * 0.32 &&
src/input/Gestu ...         bottomLeft.y - topRight.y >= bounds.height * 0.36 &&
src/input/Gestu ...         bottomLeft.x <= bounds.minX + bounds.width * 0.43 &&
src/input/Gestu ...         bottomLeft.y >= bounds.maxY - bounds.height * 0.43 &&
src/input/Gestu ...         last.x - bottomLeft.x >= bounds.width * 0.4 &&
src/input/Gestu ...         Math.abs(last.y - bottomLeft.y) <= bounds.height * 0.4
src/input/Gestu ...       ) {
src/input/Gestu ...         return true;
src/input/Gestu ...       }
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return false;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function recognizeZShape(points, bounds) {
src/input/Gestu ...   if (points.length < 4 || bounds.width < 28 || bounds.height < 28) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (followsZDirection(points, bounds) || followsZDirection(points.slice().reverse(), bounds)) {
src/input/Gestu ...     return SYMBOLS.Z;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return SYMBOLS.UNKNOWN;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function followsMDirection(points, bounds) {
src/input/Gestu ...   const first = points[0];
src/input/Gestu ...   const last = points[points.length - 1];
src/input/Gestu ...   if (
src/input/Gestu ...     first.x > bounds.minX + bounds.width * 0.34 ||
src/input/Gestu ...     first.y < bounds.maxY - bounds.height * 0.45 ||
src/input/Gestu ...     last.x < bounds.maxX - bounds.width * 0.34 ||
src/input/Gestu ...     last.y < bounds.maxY - bounds.height * 0.45
src/input/Gestu ...   ) {
src/input/Gestu ...     return false;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   for (let leftIndex = 1; leftIndex < points.length - 2; leftIndex += 1) {
src/input/Gestu ...     const leftPeak = points[leftIndex];
src/input/Gestu ...     if (
src/input/Gestu ...       leftPeak.x > bounds.minX + bounds.width * 0.38 ||
```

```
src/input/Gestu ...       leftPeak.y > bounds.minY + bounds.height * 0.35 ||
src/input/Gestu ...       first.y - leftPeak.y < bounds.height * 0.38
src/input/Gestu ...      ) {
src/input/Gestu ...       continue;
src/input/Gestu ...      }
src/input/Gestu ...
src/input/Gestu ...      for (let valleyIndex = leftIndex + 1; valleyIndex < points.length - 1; valleyIndex += 1) {
src/input/Gestu ...       const valley = points[valleyIndex];
src/input/Gestu ...       if (
src/input/Gestu ...         valley.x < bounds.minX + bounds.width * 0.25 ||
src/input/Gestu ...         valley.x > bounds.maxX - bounds.width * 0.25 ||
src/input/Gestu ...         valley.y - leftPeak.y < bounds.height * 0.3
src/input/Gestu ...       ) {
src/input/Gestu ...         continue;
src/input/Gestu ...       }
src/input/Gestu ...
src/input/Gestu ...       for (let rightIndex = valleyIndex + 1; rightIndex < points.length - 1; rightIndex += 1) {
src/input/Gestu ...         const rightPeak = points[rightIndex];
src/input/Gestu ...         if (
src/input/Gestu ...           rightPeak.x < bounds.maxX - bounds.width * 0.38 ||
src/input/Gestu ...           rightPeak.y > bounds.minY + bounds.height * 0.35 ||
src/input/Gestu ...           valley.y - rightPeak.y < bounds.height * 0.3 ||
src/input/Gestu ...           last.y - rightPeak.y < bounds.height * 0.38
src/input/Gestu ...         ) {
src/input/Gestu ...           continue;
src/input/Gestu ...         }
src/input/Gestu ...
src/input/Gestu ...         return true;
src/input/Gestu ...       }
src/input/Gestu ...      }
src/input/Gestu ...    }
src/input/Gestu ...
src/input/Gestu ...    return false;
src/input/Gestu ...  }
src/input/Gestu ...
src/input/Gestu ...  function recognizeMShape(points, bounds) {
src/input/Gestu ...    if (points.length < 5 || bounds.width < 30 || bounds.height < 28) {
src/input/Gestu ...      return SYMBOLS.UNKNOWN;
src/input/Gestu ...    }
src/input/Gestu ...
src/input/Gestu ...    if (followsMDirection(points, bounds) || followsMDirection(points.slice().reverse(), bounds)) {
src/input/Gestu ...      return SYMBOLS.M;
src/input/Gestu ...    }
src/input/Gestu ...
src/input/Gestu ...    return SYMBOLS.UNKNOWN;
src/input/Gestu ...  }
src/input/Gestu ...
src/input/Gestu ...  function followsSDirection(points, bounds) {
src/input/Gestu ...    const first = points[0];
src/input/Gestu ...    const last = points[points.length - 1];
```

```
src/input/Gestu ...
src/input/Gestu ...   if (
src/input/Gestu ...     first.x < bounds.maxX - bounds.width * 0.42 ||
src/input/Gestu ...     first.y > bounds.minY + bounds.height * 0.4 ||
src/input/Gestu ...     last.x > bounds.minX + bounds.width * 0.42 ||
src/input/Gestu ...     last.y < bounds.maxY - bounds.height * 0.38
src/input/Gestu ...   ) {
src/input/Gestu ...     return false;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   let topLeftIndex = -1;
src/input/Gestu ...   for (let i = 1; i < points.length - 2; i += 1) {
src/input/Gestu ...     const p = points[i];
src/input/Gestu ...     if (
src/input/Gestu ...       p.x <= bounds.minX + bounds.width * 0.4 &&
src/input/Gestu ...       p.y <= bounds.minY + bounds.height * 0.42
src/input/Gestu ...     ) {
src/input/Gestu ...       topLeftIndex = i;
src/input/Gestu ...       break;
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (topLeftIndex < 0) {
src/input/Gestu ...     return false;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   let middleRightIndex = -1;
src/input/Gestu ...   for (let i = topLeftIndex + 1; i < points.length - 1; i += 1) {
src/input/Gestu ...     const p = points[i];
src/input/Gestu ...     if (
src/input/Gestu ...       p.x >= bounds.maxX - bounds.width * 0.38 &&
src/input/Gestu ...       p.y >= bounds.minY + bounds.height * 0.3 &&
src/input/Gestu ...       p.y <= bounds.maxY - bounds.height * 0.2
src/input/Gestu ...     ) {
src/input/Gestu ...       middleRightIndex = i;
src/input/Gestu ...       break;
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (middleRightIndex < 0) {
src/input/Gestu ...     return false;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   for (let i = middleRightIndex + 1; i < points.length; i += 1) {
src/input/Gestu ...     const p = points[i];
src/input/Gestu ...     if (
src/input/Gestu ...       p.x <= bounds.minX + bounds.width * 0.42 &&
src/input/Gestu ...       p.y >= bounds.maxY - bounds.height * 0.38
src/input/Gestu ...     ) {
src/input/Gestu ...       return true;
```

```
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return false;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function recognizeSShape(points, bounds) {
src/input/Gestu ...   if (points.length < 6 || bounds.width < 30 || bounds.height < 36) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (followsSDirection(points, bounds) || followsSDirection(points.slice().reverse(), bounds)) {
src/input/Gestu ...     return SYMBOLS.S;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return SYMBOLS.UNKNOWN;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function followsLDirection(points, bounds) {
src/input/Gestu ...   const first = points[0];
src/input/Gestu ...   const last = points[points.length - 1];
src/input/Gestu ...   if (
src/input/Gestu ...     first.x > bounds.minX + bounds.width * 0.43 ||
src/input/Gestu ...     first.y > bounds.minY + bounds.height * 0.43 ||
src/input/Gestu ...     last.x < bounds.maxX - bounds.width * 0.43 ||
src/input/Gestu ...     last.y < bounds.maxY - bounds.height * 0.43
src/input/Gestu ...   ) {
src/input/Gestu ...     return false;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   for (let cornerIndex = 1; cornerIndex < points.length - 1; cornerIndex += 1) {
src/input/Gestu ...     const corner = points[cornerIndex];
src/input/Gestu ...     if (
src/input/Gestu ...       corner.y - first.y < bounds.height * 0.4 ||
src/input/Gestu ...       Math.abs(corner.x - first.x) > bounds.width * 0.4 ||
src/input/Gestu ...       corner.x > bounds.minX + bounds.width * 0.43 ||
src/input/Gestu ...       corner.y < bounds.maxY - bounds.height * 0.43
src/input/Gestu ...     ) {
src/input/Gestu ...       continue;
src/input/Gestu ...     }
src/input/Gestu ...
src/input/Gestu ...     if (
src/input/Gestu ...       last.x - corner.x >= bounds.width * 0.34 &&
src/input/Gestu ...       Math.abs(last.y - corner.y) <= bounds.height * 0.4
src/input/Gestu ...     ) {
src/input/Gestu ...       return true;
src/input/Gestu ...     }
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return false;
```

```
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function recognizeLShape(points, bounds) {
src/input/Gestu ...   if (points.length < 3 || bounds.width < 24 || bounds.height < 28) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   if (followsLDirection(points, bounds) || followsLDirection(points.slice().reverse(), bounds)) {
src/input/Gestu ...     return SYMBOLS.L;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return SYMBOLS.UNKNOWN;
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... function recognize(points) {
src/input/Gestu ...   const simplified = simplify(points);
src/input/Gestu ...   if (simplified.length < 2) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const bounds = getBounds(simplified);
src/input/Gestu ...   if (pathLength(simplified) < 24) {
src/input/Gestu ...     return SYMBOLS.UNKNOWN;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const circle = recognizeCircle(simplified, bounds);
src/input/Gestu ...   if (circle !== SYMBOLS.UNKNOWN) {
src/input/Gestu ...     return circle;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const mShape = recognizeMShape(simplified, bounds);
src/input/Gestu ...   if (mShape !== SYMBOLS.UNKNOWN) {
src/input/Gestu ...     return mShape;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const zShape = recognizeZShape(simplified, bounds);
src/input/Gestu ...   if (zShape !== SYMBOLS.UNKNOWN) {
src/input/Gestu ...     return zShape;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const sShape = recognizeSShape(simplified, bounds);
src/input/Gestu ...   if (sShape !== SYMBOLS.UNKNOWN) {
src/input/Gestu ...     return sShape;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   const lShape = recognizeLShape(simplified, bounds);
src/input/Gestu ...   if (lShape !== SYMBOLS.UNKNOWN) {
src/input/Gestu ...     return lShape;
src/input/Gestu ...   }
src/input/Gestu ...
```

```
src/input/Gestu ...   const vShape = recognizeVShape(simplified, bounds);
src/input/Gestu ...   if (vShape !== SYMBOLS.UNKNOWN) {
src/input/Gestu ...     return vShape;
src/input/Gestu ...   }
src/input/Gestu ...
src/input/Gestu ...   return recognizeLine(simplified, bounds);
src/input/Gestu ... }
src/input/Gestu ...
src/input/Gestu ... module.exports = {
src/input/Gestu ...   SYMBOLS,
src/input/Gestu ...   LABELS,
src/input/Gestu ...   recognize,
src/input/Gestu ...   simplify,
src/input/Gestu ...   getBounds,
src/input/Gestu ...   pathLength
src/input/Gestu ... };
src/input/Gestu ...
src/render/Ren ... // ===== 文件：src/render/Renderer.js =====
src/render/Ren ... 'use strict';
src/render/Ren ...
src/render/Ren ... const { LABELS, SYMBOLS } = require('../input/GestureRecognizer');
src/render/Ren ... const { SCREENS } = require('../core/GameState');
src/render/Ren ... const { THEME_IDS, getTheme } = require('../levels/Themes');
src/render/Ren ... const { DIFFICULTY_MODES, getDifficultyButtons } = require('../ui/DifficultySelector');
src/render/Ren ... const { getAudioToggleBounds } = require('../ui/AudioToggle');
src/render/Ren ... const { ITEM_TYPES, getHeartSlotPosition, getItemSlots } = require('../ui/ItemBar');
src/render/Ren ...
src/render/Ren ... const ENEMY_ASSET_KEYS = {
src/render/Ren ...   jellyfish: 'enemyJellyfish',
src/render/Ren ...   pufferfish: 'enemyPufferfish',
src/render/Ren ...   shark: 'enemyShark',
src/render/Ren ...   megalodon: 'bossMegalodon',
src/render/Ren ...   penguinBellhop: 'enemyPenguinBellhop',
src/render/Ren ...   penguinChef: 'enemyPenguinChef',
src/render/Ren ...   emperorPenguin: 'bossEmperorPenguin',
src/render/Ren ...   pterosaur: 'enemyPterosaur',
src/render/Ren ...   triceratops: 'enemyTriceratops',
src/render/Ren ...   brachiosaurus: 'enemyBrachiosaurus',
src/render/Ren ...   tyrannosaurus: 'bossTyrannosaurus'
src/render/Ren ... };
src/render/Ren ...
src/render/Ren ... const SYMBOL_ICON_ASSET_KEYS = {
src/render/Ren ...   [SYMBOLS.UP]: 'symbolUp',
src/render/Ren ...   [SYMBOLS.DOWN]: 'symbolDown',
src/render/Ren ...   [SYMBOLS.LEFT]: 'symbolLeft',
src/render/Ren ...   [SYMBOLS.RIGHT]: 'symbolRight',
src/render/Ren ...   [SYMBOLS.V]: 'symbolV',
src/render/Ren ...   [SYMBOLS.L]: 'symbolL',
src/render/Ren ...   [SYMBOLS.CIRCLE]: 'symbolCircle',
src/render/Ren ...   [SYMBOLS.Z]: 'symbolZ',
```

```
src/render/Ren ...     [SYMBOLS.M]: 'symbolM',
src/render/Ren ...     [SYMBOLS.S]: 'symbolS'
src/render/Ren ...   };
src/render/Ren ...
src/render/Ren ...   const ENEMY_SPRITE = {
src/render/Ren ...     frames: 3,
src/render/Ren ...     fps: 6,
src/render/Ren ...     phaseSpeed: 4
src/render/Ren ...   };
src/render/Ren ...
src/render/Ren ...   function getEnemySpriteFrame(phase) {
src/render/Ren ...     const animationTime = (phase || 0) / ENEMY_SPRITE.phaseSpeed;
src/render/Ren ...     return Math.floor(animationTime * ENEMY_SPRITE.fps) % ENEMY_SPRITE.frames;
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   const HERO_SPRITE = {
src/render/Ren ...     frameWidth: 128,
src/render/Ren ...     frameHeight: 128,
src/render/Ren ...     animations: {
src/render/Ren ...       walk: {
src/render/Ren ...         assetKey: 'catWalk',
src/render/Ren ...         frames: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
src/render/Ren ...         fps: 12
src/render/Ren ...       },
src/render/Ren ...       cast: {
src/render/Ren ...         assetKey: 'catCast',
src/render/Ren ...         frames: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
src/render/Ren ...         fps: 16
src/render/Ren ...       },
src/render/Ren ...       hurt: {
src/render/Ren ...         assetKey: 'catHurt',
src/render/Ren ...         frames: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11],
src/render/Ren ...         fps: 14
src/render/Ren ...       }
src/render/Ren ...     }
src/render/Ren ...   };
src/render/Ren ...
src/render/Ren ...   function getHeroSpriteAnimation(animation, isDrawing) {
src/render/Ren ...     if (animation && animation.hurt > 0) {
src/render/Ren ...       return 'hurt';
src/render/Ren ...     }
src/render/Ren ...     if (isDrawing || (animation && animation.cast > 0)) {
src/render/Ren ...       return 'cast';
src/render/Ren ...     }
src/render/Ren ...     return 'walk';
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   function getHeroSpriteFrame(elapsed, animation, isDrawing) {
src/render/Ren ...     const animationName = getHeroSpriteAnimation(animation, isDrawing);
src/render/Ren ...     const track = HERO_SPRITE.animations[animationName];
```

```
src/render/Ren ...   let animationTime = elapsed || 0;
src/render/Ren ...
src/render/Ren ...   if (animationName === 'hurt') {
src/render/Ren ...     animationTime = animation.hurtAge || 0;
src/render/Ren ...   } else if (animationName === 'cast' && animation && animation.cast > 0) {
src/render/Ren ...     animationTime = animation.castAge || 0;
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   const frameIndex = Math.floor(animationTime * track.fps) % track.frames.length;
src/render/Ren ...   return track.frames[frameIndex];
src/render/Ren ... }
src/render/Ren ...
src/render/Ren ... function getSymbolIndicators(enemy) {
src/render/Ren ...   if (enemy.symbolDisplay !== 'current-and-dots') {
src/render/Ren ...     return enemy.symbols.map((symbol) => ({ type: 'symbol', symbol }));
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   if (enemy.symbols.length > 5) {
src/render/Ren ...     return [
src/render/Ren ...       { type: 'symbol', symbol: enemy.symbols[0] },
src/render/Ren ...       { type: 'count', count: enemy.symbols.length - 1 }
src/render/Ren ...     ];
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   return enemy.symbols.map((symbol, index) => (
src/render/Ren ...     index === 0 ? { type: 'symbol', symbol } : { type: 'dot' }
src/render/Ren ...   ));
src/render/Ren ... }
src/render/Ren ...
src/render/Ren ... function getLevelLabel(state, visibleLevel) {
src/render/Ren ...   const difficultyLabel = state.screen !== SCREENS.LEVEL_TRANSITION && state.difficulty === DIFFICULTY_MODES.PLUS_ONE ? '  +1'  ...
src/render/Ren ...   return '第 ' + (visibleLevel || state.level) + '/' + state.totalLevels + ' 关' + difficultyLabel;
src/render/Ren ... }
src/render/Ren ...
src/render/Ren ... function getComboLabel(combo) {
src/render/Ren ...   return combo > 0 ? '连击 x' + combo : '';
src/render/Ren ... }
src/render/Ren ...
src/render/Ren ... function getComboTier(combo) {
src/render/Ren ...   if (combo >= 100) {
src/render/Ren ...     return {
src/render/Ren ...       color: '#ffc2ff',
src/render/Ren ...       glow: 'rgba(255, 105, 255, 0.92)',
src/render/Ren ...       background: 'rgba(86, 15, 94, 0.78)',
src/render/Ren ...       multiplierLabel: '4.0x',
src/render/Ren ...       scale: 1.44
src/render/Ren ...     };
src/render/Ren ...   }
src/render/Ren ...   if (combo >= 50) {
src/render/Ren ...     return {
```

```
src/render/Ren ...      ctx.closePath();
src/render/Ren ...      ctx.fill();
src/render/Ren ...
src/render/Ren ...      ctx.fillStyle = '#f0c36c';
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.roundRect(-r * 0.22, -r * 1.3, r * 0.44, r * 0.12, [r * 0.03]);
src/render/Ren ...      ctx.fill();
src/render/Ren ...
src/render/Ren ...      ctx.fillStyle = '#ffcb74';
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.arc(-r * 0.18, -r * 0.48, r * 0.07, 0, Math.PI * 2);
src/render/Ren ...      ctx.arc(r * 0.18, -r * 0.48, r * 0.07, 0, Math.PI * 2);
src/render/Ren ...      ctx.fill();
src/render/Ren ...
src/render/Ren ...      ctx.strokeStyle = '#684949';
src/render/Ren ...      ctx.lineWidth = 1.5;
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.moveTo(-r * 0.2, -r * 0.22);
src/render/Ren ...      ctx.quadraticCurveTo(0, -r * 0.12, r * 0.2, -r * 0.22);
src/render/Ren ...      ctx.stroke();
src/render/Ren ...
src/render/Ren ...      ctx.strokeStyle = '#9edbe8';
src/render/Ren ...      ctx.lineWidth = 3;
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.moveTo(-r * 0.72, r * 0.22);
src/render/Ren ...      ctx.lineTo(r * 0.72, r * 0.22);
src/render/Ren ...      ctx.moveTo(-r * 0.45, r * 0.54);
src/render/Ren ...      ctx.lineTo(r * 0.45, r * 0.54);
src/render/Ren ...      ctx.stroke();
src/render/Ren ...
src/render/Ren ...      ctx.fillStyle = 'rgba(255, 212, 129, 0.8)';
src/render/Ren ...      for (let i = 0; i < 3; i += 1) {
src/render/Ren ...        ctx.beginPath();
src/render/Ren ...        ctx.roundRect(-r * 0.42 + i * r * 0.28, r * 0.02, r * 0.18, r * 0.14, [r * 0.03]);
src/render/Ren ...        ctx.fill();
src/render/Ren ...      }
src/render/Ren ...
src/render/Ren ...      ctx.fillStyle = '#d15d5d';
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.moveTo(-r * 0.24, -r * 0.08);
src/render/Ren ...      ctx.lineTo(0, r * 0.12);
src/render/Ren ...      ctx.lineTo(r * 0.24, -r * 0.08);
src/render/Ren ...      ctx.closePath();
src/render/Ren ...      ctx.fill();
src/render/Ren ...
src/render/Ren ...      ctx.strokeStyle = 'rgba(255, 211, 132, 0.7)';
src/render/Ren ...      ctx.lineWidth = 2;
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.arc(0, r * 0.18, r * 0.5, Math.PI * 0.08, Math.PI * 0.92);
src/render/Ren ...      ctx.stroke();
```

```
src/render/Ren ...
src/render/Ren ...    ctx.strokeStyle = 'rgba(255, 211, 132, 0.32)';
src/render/Ren ...    ctx.lineWidth = 5;
src/render/Ren ...    ctx.beginPath();
src/render/Ren ...    ctx.arc(0, -r * 0.1, r * 0.82, Math.PI * 0.12, Math.PI * 0.88);
src/render/Ren ...    ctx.stroke();
src/render/Ren ...    ctx.restore();
src/render/Ren ...  }
src/render/Ren ...
src/render/Ren ...  drawSymbolQueue(enemy) {
src/render/Ren ...    const ctx = this.ctx;
src/render/Ren ...    const indicators = getSymbolIndicators(enemy);
src/render/Ren ...    const count = indicators.length;
src/render/Ren ...    const size = Math.max(30, enemy.radius * 0.78);
src/render/Ren ...    const gap = 5;
src/render/Ren ...    const startX = -((count * size + (count - 1) * gap) / 2) + size / 2;
src/render/Ren ...    const imageScale = enemy.kind === 'boss' ? 1.62 : 1.36;
src/render/Ren ...    const visualRadius = enemy.species === 'ghost' ? enemy.radius : enemy.radius * imageScale;
src/render/Ren ...    const y = -visualRadius - 24;
src/render/Ren ...
src/render/Ren ...    ctx.font = 'bold ' + Math.floor(size * 0.82) + 'px sans-serif';
src/render/Ren ...    ctx.textAlign = 'center';
src/render/Ren ...    ctx.textBaseline = 'middle';
src/render/Ren ...
src/render/Ren ...    for (let i = 0; i < count; i += 1) {
src/render/Ren ...      const x = startX + i * (size + gap);
src/render/Ren ...      const indicator = indicators[i];
src/render/Ren ...      if (indicator.type === 'dot') {
src/render/Ren ...        ctx.fillStyle = 'rgba(217, 246, 255, 0.84)';
src/render/Ren ...        ctx.beginPath();
src/render/Ren ...        ctx.arc(x, y, Math.max(4, size * 0.16), 0, Math.PI * 2);
src/render/Ren ...        ctx.fill();
src/render/Ren ...        continue;
src/render/Ren ...      }
src/render/Ren ...      if (indicator.type === 'count') {
src/render/Ren ...        ctx.fillStyle = 'rgba(217, 246, 255, 0.88)';
src/render/Ren ...        ctx.beginPath();
src/render/Ren ...        ctx.roundRect(x - size / 2, y - size / 2, size, size, [7]);
src/render/Ren ...        ctx.fill();
src/render/Ren ...        ctx.fillStyle = '#1d2740';
src/render/Ren ...        ctx.font = 'bold ' + Math.floor(size * 0.58) + 'px sans-serif';
src/render/Ren ...        ctx.fillText('+' + indicator.count, x, y + 1);
src/render/Ren ...        ctx.font = 'bold ' + Math.floor(size * 0.82) + 'px sans-serif';
src/render/Ren ...        continue;
src/render/Ren ...      }
src/render/Ren ...      ctx.fillStyle = i === 0 ? '#ffdc6f' : 'rgba(255,255,255,0.68)';
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.roundRect(x - size / 2, y - size / 2, size, size, [7]);
src/render/Ren ...      ctx.fill();
src/render/Ren ...      const iconAssetKey = SYMBOL_ICON_ASSET_KEYS[indicator.symbol];
```

```
src/render/Ren ...       const symbolIcon = iconAssetKey && this.assets && this.assets.getImage(iconAssetKey);
src/render/Ren ...       if (symbolIcon) {
src/render/Ren ...         const iconSize = size * 0.9;
src/render/Ren ...         ctx.drawImage(
src/render/Ren ...           symbolIcon,
src/render/Ren ...           x - iconSize / 2,
src/render/Ren ...           y - iconSize / 2,
src/render/Ren ...           iconSize,
src/render/Ren ...           iconSize
src/render/Ren ...         );
src/render/Ren ...       } else {
src/render/Ren ...         ctx.fillStyle = '#1d2740';
src/render/Ren ...         ctx.fillText(LABELS[indicator.symbol], x, y + 1);
src/render/Ren ...       }
src/render/Ren ...     }
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawGesture(path) {
src/render/Ren ...     if (!path || path.length < 2) {
src/render/Ren ...       return;
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.strokeStyle = '#8bf2ff';
src/render/Ren ...     ctx.lineWidth = 6;
src/render/Ren ...     ctx.lineCap = 'round';
src/render/Ren ...     ctx.lineJoin = 'round';
src/render/Ren ...     ctx.shadowColor = '#8bf2ff';
src/render/Ren ...     ctx.shadowBlur = 10;
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.moveTo(path[0].x, path[0].y);
src/render/Ren ...     for (let i = 1; i < path.length; i += 1) {
src/render/Ren ...       ctx.lineTo(path[i].x, path[i].y);
src/render/Ren ...     }
src/render/Ren ...     ctx.stroke();
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawHud(state, visibleLevel) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.fillStyle = 'rgba(8, 13, 25, 0.32)';
src/render/Ren ...     ctx.fillRect(0, 0, this.width, 62);
src/render/Ren ...
src/render/Ren ...     ctx.fillStyle = '#fff2c2';
src/render/Ren ...     ctx.font = 'bold 20px sans-serif';
src/render/Ren ...     ctx.textAlign = 'left';
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     ctx.fillText('分数 ' + state.score, 16, 31);
src/render/Ren ...
```

```
src/render/Ren ...     ctx.fillStyle = '#d8e6ff';
src/render/Ren ...     ctx.font = 'bold 13px sans-serif';
src/render/Ren ...     ctx.textAlign = 'center';
src/render/Ren ...     const themeId = state.getThemeId ? state.getThemeId(visibleLevel) : THEME_IDS.CASTLE;
src/render/Ren ...     ctx.fillText(getLevelLabel(state, visibleLevel) + ' · ' + getTheme(themeId).name, this.width * 0.5, 31);
src/render/Ren ...
src/render/Ren ...     this.drawComboCounter(state);
src/render/Ren ...     this.drawHeartBar(state);
src/render/Ren ...     this.drawItemBar(state);
src/render/Ren ...
src/render/Ren ...     if (state.feedback) {
src/render/Ren ...       ctx.globalAlpha = Math.max(0, 1 - state.feedback.age / 0.7);
src/render/Ren ...       ctx.fillStyle = state.feedback.type === 'hit' ? '#a4ffbf' : (state.feedback.type === 'item' ? '#fff2c2' : '#ff9a87');
src/render/Ren ...       ctx.font = 'bold 24px sans-serif';
src/render/Ren ...       ctx.textAlign = 'center';
src/render/Ren ...       ctx.fillText(state.feedback.text, this.width * 0.5, 104 - state.feedback.age * 24);
src/render/Ren ...       ctx.globalAlpha = 1;
src/render/Ren ...     }
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawComboCounter(state) {
src/render/Ren ...     if (state.screen !== SCREENS.PLAYING || state.combo <= 0) {
src/render/Ren ...       return;
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     const tier = getComboTier(state.combo);
src/render/Ren ...     const pulse = getComboPulse(state.combo, state.feedback);
src/render/Ren ...     const label = getComboLabel(state.combo);
src/render/Ren ...     const x = 16;
src/render/Ren ...     const y = 85 - pulse.lift;
src/render/Ren ...
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.font = 'bold 17px sans-serif';
src/render/Ren ...     ctx.textAlign = 'left';
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     const labelWidth = ctx.measureText(label).width;
src/render/Ren ...     const multiplierWidth = tier.multiplierLabel ? 45 : 0;
src/render/Ren ...     const width = labelWidth + multiplierWidth + 18;
src/render/Ren ...
src/render/Ren ...     ctx.translate(x, y);
src/render/Ren ...     ctx.scale(tier.scale * pulse.scale, tier.scale * pulse.scale);
src/render/Ren ...
src/render/Ren ...     if (pulse.burst > 0) {
src/render/Ren ...       const spread = (1 - pulse.burst) * 13;
src/render/Ren ...       ctx.globalAlpha = pulse.burst * 0.72;
src/render/Ren ...       ctx.strokeStyle = tier.glow;
src/render/Ren ...       ctx.lineWidth = 1.5 + pulse.burst * 2.5;
src/render/Ren ...       ctx.beginPath();
src/render/Ren ...       ctx.roundRect(-spread, -16 - spread * 0.38, width + spread * 2, 32 + spread * 0.76, [13]);
```

```
src/render/Ren ...      ctx.stroke();
src/render/Ren ...      ctx.globalAlpha = 1;
src/render/Ren ...    }
src/render/Ren ...
src/render/Ren ...    ctx.fillStyle = tier.background;
src/render/Ren ...    ctx.beginPath();
src/render/Ren ...    ctx.roundRect(0, -14, width, 28, [11]);
src/render/Ren ...    ctx.fill();
src/render/Ren ...
src/render/Ren ...    ctx.strokeStyle = tier.glow;
src/render/Ren ...    ctx.globalAlpha = 0.48 + pulse.burst * 0.45;
src/render/Ren ...    ctx.lineWidth = 1;
src/render/Ren ...    ctx.stroke();
src/render/Ren ...    ctx.globalAlpha = 1;
src/render/Ren ...
src/render/Ren ...    ctx.shadowColor = tier.glow;
src/render/Ren ...    ctx.shadowBlur = 5 + pulse.burst * 11;
src/render/Ren ...    ctx.fillStyle = tier.color;
src/render/Ren ...    ctx.fillText(label, 9, 0);
src/render/Ren ...
src/render/Ren ...    if (tier.multiplierLabel) {
src/render/Ren ...      ctx.shadowBlur = 0;
src/render/Ren ...      ctx.fillStyle = 'rgba(255, 255, 255, 0.17)';
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.roundRect(width - 42, -10, 36, 20, [7]);
src/render/Ren ...      ctx.fill();
src/render/Ren ...      ctx.font = 'bold 12px sans-serif';
src/render/Ren ...      ctx.textAlign = 'center';
src/render/Ren ...      ctx.fillStyle = tier.color;
src/render/Ren ...      ctx.fillText(tier.multiplierLabel, width - 24, 0);
src/render/Ren ...    }
src/render/Ren ...
src/render/Ren ...    ctx.restore();
src/render/Ren ...  }
src/render/Ren ...
src/render/Ren ...  drawHeartBar(state) {
src/render/Ren ...    for (let i = 0; i < 5; i += 1) {
src/render/Ren ...      const position = getHeartSlotPosition(this.width, this.height, i);
src/render/Ren ...      this.drawHeart(position.x, position.y, i < state.lives);
src/render/Ren ...    }
src/render/Ren ...  }
src/render/Ren ...
src/render/Ren ...  drawItemBar(state) {
src/render/Ren ...    if (state.screen !== SCREENS.PLAYING) {
src/render/Ren ...      return;
src/render/Ren ...    }
src/render/Ren ...
src/render/Ren ...    const slots = getItemSlots(this.width, this.height, state.items);
src/render/Ren ...    for (let i = 0; i < slots.length; i += 1) {
src/render/Ren ...      this.drawItemSlot(slots[i], state.elapsed || 0);
```

```
src/render/Ren ...     }
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawItemSlot(slot, elapsed) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     const centerX = slot.x + slot.width * 0.5;
src/render/Ren ...     const centerY = slot.y + slot.height * 0.5;
src/render/Ren ...
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.fillStyle = 'rgba(8, 13, 25, 0.68)';
src/render/Ren ...     ctx.strokeStyle = slot.type === ITEM_TYPES.COMBO_CHAIN ? '#d7a2ff' : '#ff8fa8';
src/render/Ren ...     ctx.lineWidth = 1.5;
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.roundRect(slot.x, slot.y, slot.width, slot.height, [13]);
src/render/Ren ...     ctx.fill();
src/render/Ren ...     ctx.stroke();
src/render/Ren ...     ctx.translate(centerX, centerY);
src/render/Ren ...     if (slot.type === ITEM_TYPES.HEALTH_POTION) {
src/render/Ren ...       this.drawHealthPotion({ radius: 18, phase: elapsed * 4 });
src/render/Ren ...     } else {
src/render/Ren ...       this.drawComboChainItemIcon(20);
src/render/Ren ...     }
src/render/Ren ...     ctx.restore();
src/render/Ren ...
src/render/Ren ...     const label = slot.count > 99 ? '99+' : String(slot.count);
src/render/Ren ...     const badgeWidth = label.length > 2 ? 28 : 21;
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.fillStyle = '#fff2c2';
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.roundRect(slot.x + slot.width - badgeWidth + 5, slot.y + 4, badgeWidth, 18, [9]);
src/render/Ren ...     ctx.fill();
src/render/Ren ...     ctx.fillStyle = '#38224a';
src/render/Ren ...     ctx.font = 'bold 12px sans-serif';
src/render/Ren ...     ctx.textAlign = 'center';
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     ctx.fillText(label, slot.x + slot.width - badgeWidth * 0.5 + 5, slot.y + 13);
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawItemEarnEffects(effects) {
src/render/Ren ...     if (!effects) {
src/render/Ren ...       return;
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     for (let i = 0; i < effects.length; i += 1) {
src/render/Ren ...       if (effects[i].type === 'itemEarn') {
src/render/Ren ...         this.drawItemEarnEffect(effects[i]);
src/render/Ren ...       }
src/render/Ren ...     }
src/render/Ren ...   }
```

```
src/render/Ren ...
src/render/Ren ...   drawItemEarnEffect(effect) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     const progress = Math.max(0, Math.min(1, effect.age / effect.duration));
src/render/Ren ...     const eased = 1 - Math.pow(1 - progress, 3);
src/render/Ren ...     const fromX = 52;
src/render/Ren ...     const fromY = 85;
src/render/Ren ...     const arcLift = Math.sin(progress * Math.PI) * Math.max(34, this.height * 0.09);
src/render/Ren ...     const x = fromX + (effect.toX - fromX) * eased;
src/render/Ren ...     const y = fromY + (effect.toY - fromY) * eased - arcLift;
src/render/Ren ...     const scale = 1.1 - eased * 0.28;
src/render/Ren ...     const arrival = Math.max(0, (progress - 0.72) / 0.28);
src/render/Ren ...
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.translate(x, y);
src/render/Ren ...     ctx.scale(scale, scale);
src/render/Ren ...     ctx.globalAlpha = Math.max(0, 1 - Math.max(0, progress - 0.92) / 0.08);
src/render/Ren ...     if (effect.itemType === ITEM_TYPES.HEALTH_POTION) {
src/render/Ren ...       this.drawHealthPotion({ radius: 17, phase: progress * Math.PI * 4 });
src/render/Ren ...     } else {
src/render/Ren ...       this.drawComboChainItemIcon(19);
src/render/Ren ...     }
src/render/Ren ...     ctx.restore();
src/render/Ren ...
src/render/Ren ...     if (arrival > 0) {
src/render/Ren ...       ctx.save();
src/render/Ren ...       ctx.translate(effect.toX, effect.toY);
src/render/Ren ...       ctx.globalAlpha = 1 - arrival;
src/render/Ren ...       ctx.strokeStyle = effect.itemType === ITEM_TYPES.COMBO_CHAIN ? '#d7a2ff' : '#ff8fa8';
src/render/Ren ...       ctx.lineWidth = 2.4;
src/render/Ren ...       ctx.shadowColor = ctx.strokeStyle;
src/render/Ren ...       ctx.shadowBlur = 14;
src/render/Ren ...       ctx.beginPath();
src/render/Ren ...       ctx.arc(0, 0, 18 + arrival * 16, 0, Math.PI * 2);
src/render/Ren ...       ctx.stroke();
src/render/Ren ...       ctx.restore();
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...   drawComboChainItemIcon(radius) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.strokeStyle = '#ead2ff';
src/render/Ren ...     ctx.lineWidth = 5;
src/render/Ren ...     ctx.lineCap = 'round';
src/render/Ren ...     ctx.lineJoin = 'round';
src/render/Ren ...     ctx.shadowColor = '#ca78ff';
src/render/Ren ...     ctx.shadowBlur = 15;
src/render/Ren ...     ctx.beginPath();
```

```
src/render/Ren ...     ctx.moveTo(radius * 0.2, -radius * 0.95);
src/render/Ren ...     ctx.lineTo(-radius * 0.46, -radius * 0.08);
src/render/Ren ...     ctx.lineTo(radius * 0.04, -radius * 0.08);
src/render/Ren ...     ctx.lineTo(-radius * 0.24, radius * 0.96);
src/render/Ren ...     ctx.lineTo(radius * 0.68, -radius * 0.23);
src/render/Ren ...     ctx.lineTo(radius * 0.18, -radius * 0.23);
src/render/Ren ...     ctx.stroke();
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawHeart(x, y, filled) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.translate(x, y);
src/render/Ren ...     ctx.fillStyle = filled ? '#ff6f91' : 'rgba(255,255,255,0.25)';
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.moveTo(0, 8);
src/render/Ren ...     ctx.bezierCurveTo(-16, -4, -8, -18, 0, -8);
src/render/Ren ...     ctx.bezierCurveTo(8, -18, 16, -4, 0, 8);
src/render/Ren ...     ctx.fill();
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawDifficultyOverlay(title, subtitle, prompt) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.fillStyle = 'rgba(8, 13, 25, 0.7)';
src/render/Ren ...     ctx.fillRect(0, 0, this.width, this.height);
src/render/Ren ...
src/render/Ren ...     ctx.textAlign = 'center';
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     ctx.fillStyle = '#fff2c2';
src/render/Ren ...     ctx.font = 'bold 40px sans-serif';
src/render/Ren ...     ctx.fillText(title, this.width * 0.5, this.height * 0.29);
src/render/Ren ...
src/render/Ren ...     ctx.fillStyle = '#d8e6ff';
src/render/Ren ...     ctx.font = '20px sans-serif';
src/render/Ren ...     ctx.fillText(subtitle, this.width * 0.5, this.height * 0.38);
src/render/Ren ...
src/render/Ren ...     ctx.fillStyle = '#ffdc6f';
src/render/Ren ...     ctx.font = 'bold 18px sans-serif';
src/render/Ren ...     ctx.fillText(prompt, this.width * 0.5, this.height * 0.46);
src/render/Ren ...
src/render/Ren ...     const buttons = getDifficultyButtons(this.width, this.height);
src/render/Ren ...     for (let i = 0; i < buttons.length; i += 1) {
src/render/Ren ...       this.drawDifficultyButton(buttons[i], buttons[i].mode === DIFFICULTY_MODES.PLUS_ONE);
src/render/Ren ...     }
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawDifficultyButton(button, emphasized) {
src/render/Ren ...     const ctx = this.ctx;
```

```
src/render/Ren ...     ctx.fillStyle = emphasized ? 'rgba(255, 220, 111, 0.19)' : 'rgba(216, 230, 255, 0.1)';
src/render/Ren ...     ctx.strokeStyle = emphasized ? '#ffdc6f' : 'rgba(216, 230, 255, 0.72)';
src/render/Ren ...     ctx.lineWidth = 2;
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.roundRect(button.x, button.y, button.width, button.height, [12]);
src/render/Ren ...     ctx.fill();
src/render/Ren ...     ctx.stroke();
src/render/Ren ...
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     ctx.textAlign = 'left';
src/render/Ren ...     ctx.fillStyle = emphasized ? '#ffdc6f' : '#fff2c2';
src/render/Ren ...     ctx.font = 'bold 20px sans-serif';
src/render/Ren ...     ctx.fillText(button.label, button.x + 18, button.y + button.height * 0.5);
src/render/Ren ...
src/render/Ren ...     ctx.textAlign = 'right';
src/render/Ren ...     ctx.fillStyle = '#d8e6ff';
src/render/Ren ...     ctx.font = '15px sans-serif';
src/render/Ren ...     ctx.fillText(button.detail, button.x + button.width - 18, button.y + button.height * 0.5);
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawAudioToggle(enabled) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     const button = getAudioToggleBounds(this.width);
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.fillStyle = enabled ? 'rgba(255, 220, 111, 0.17)' : 'rgba(216, 230, 255, 0.1)';
src/render/Ren ...     ctx.strokeStyle = enabled ? '#ffdc6f' : 'rgba(216, 230, 255, 0.64)';
src/render/Ren ...     ctx.lineWidth = 1.5;
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.roundRect(button.x, button.y, button.width, button.height, [17]);
src/render/Ren ...     ctx.fill();
src/render/Ren ...     ctx.stroke();
src/render/Ren ...     ctx.fillStyle = enabled ? '#ffdc6f' : '#d8e6ff';
src/render/Ren ...     ctx.font = 'bold 14px sans-serif';
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     ctx.textAlign = 'center';
src/render/Ren ...     ctx.fillText(getSoundToggleLabel(enabled), button.x + button.width * 0.5, button.y + button.height * 0.5);
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawOverlay(title, subtitle, action, result) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.save();
src/render/Ren ...     this.drawResultBackground(result);
src/render/Ren ...
src/render/Ren ...     ctx.textAlign = 'center';
src/render/Ren ...     ctx.textBaseline = 'middle';
src/render/Ren ...     ctx.shadowColor = result === 'lose' ? 'rgba(255, 103, 96, 0.72)' : 'rgba(255, 220, 111, 0.82)';
src/render/Ren ...     ctx.shadowBlur = 18;
src/render/Ren ...     ctx.fillStyle = result === 'lose' ? '#ffe4df' : '#fff6c8';
src/render/Ren ...     ctx.font = 'bold 42px sans-serif';
```

```
src/render/Ren ...     ctx.fillText(title, this.width * 0.5, this.height * 0.34);
src/render/Ren ...
src/render/Ren ...     ctx.shadowBlur = 10;
src/render/Ren ...     ctx.fillStyle = '#d8e6ff';
src/render/Ren ...     ctx.font = '20px sans-serif';
src/render/Ren ...     ctx.fillText(subtitle, this.width * 0.5, this.height * 0.44);
src/render/Ren ...
src/render/Ren ...     ctx.shadowBlur = 14;
src/render/Ren ...     ctx.fillStyle = result === 'lose' ? '#ffb1a3' : '#ffdc6f';
src/render/Ren ...     ctx.font = 'bold 22px sans-serif';
src/render/Ren ...     ctx.fillText(action, this.width * 0.5, this.height * 0.55);
src/render/Ren ...
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawResultBackground(result) {
src/render/Ren ...     if (result === 'lose') {
src/render/Ren ...       this.drawLoseResultBackground();
src/render/Ren ...       return;
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     this.drawWinResultBackground();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawWinResultBackground() {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     const width = this.width;
src/render/Ren ...     const height = this.height;
src/render/Ren ...     ctx.save();
src/render/Ren ...
src/render/Ren ...     const sky = ctx.createLinearGradient(0, 0, 0, height);
src/render/Ren ...     sky.addColorStop(0, 'rgba(21, 49, 101, 0.94)');
src/render/Ren ...     sky.addColorStop(0.45, 'rgba(74, 134, 187, 0.9)');
src/render/Ren ...     sky.addColorStop(1, 'rgba(255, 214, 135, 0.88)');
src/render/Ren ...     ctx.fillStyle = sky;
src/render/Ren ...     ctx.fillRect(0, 0, width, height);
src/render/Ren ...
src/render/Ren ...     ctx.fillStyle = 'rgba(255, 249, 218, 0.22)';
src/render/Ren ...     for (let i = 0; i < 18; i += 1) {
src/render/Ren ...       const x = (i * 73) % width;
src/render/Ren ...       const y = height * (0.08 + ((i * 31) % 37) / 100);
src/render/Ren ...       const radius = 1.6 + (i % 4) * 0.8;
src/render/Ren ...       ctx.beginPath();
src/render/Ren ...       ctx.arc(x, y, radius, 0, Math.PI * 2);
src/render/Ren ...       ctx.fill();
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     ctx.fillStyle = 'rgba(255, 247, 215, 0.25)';
src/render/Ren ...     for (let i = 0; i < 4; i += 1) {
src/render/Ren ...       const x = width * (0.14 + i * 0.24);
```

```
src/render/Ren ...       const y = height * (0.2 + (i % 2) * 0.08);
src/render/Ren ...       ctx.beginPath();
src/render/Ren ...       ctx.ellipse(x, y, width * 0.16, height * 0.035, 0, 0, Math.PI * 2);
src/render/Ren ...       ctx.fill();
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     const portalX = width * 0.5;
src/render/Ren ...     const portalY = height * 0.67;
src/render/Ren ...     const portalRadius = Math.min(width, height) * 0.26;
src/render/Ren ...     const portal = ctx.createRadialGradient(portalX, portalY, portalRadius * 0.12, portalX, portalY, portalRadius);
src/render/Ren ...     portal.addColorStop(0, 'rgba(255, 249, 198, 0.78)');
src/render/Ren ...     portal.addColorStop(0.38, 'rgba(255, 216, 91, 0.34)');
src/render/Ren ...     portal.addColorStop(1, 'rgba(255, 216, 91, 0)');
src/render/Ren ...     ctx.fillStyle = portal;
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.arc(portalX, portalY, portalRadius, 0, Math.PI * 2);
src/render/Ren ...     ctx.fill();
src/render/Ren ...
src/render/Ren ...     ctx.strokeStyle = 'rgba(255, 238, 153, 0.78)';
src/render/Ren ...     ctx.lineWidth = 5;
src/render/Ren ...     ctx.shadowColor = '#ffdc6f';
src/render/Ren ...     ctx.shadowBlur = 24;
src/render/Ren ...     for (let i = 0; i < 3; i += 1) {
src/render/Ren ...       ctx.beginPath();
src/render/Ren ...       ctx.arc(portalX, portalY, portalRadius * (0.45 + i * 0.17), Math.PI * 0.12, Math.PI * 1.88);
src/render/Ren ...       ctx.stroke();
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     ctx.shadowBlur = 0;
src/render/Ren ...     ctx.fillStyle = 'rgba(47, 63, 107, 0.62)';
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.moveTo(0, height * 0.78);
src/render/Ren ...     ctx.quadraticCurveTo(width * 0.28, height * 0.68, width * 0.52, height * 0.79);
src/render/Ren ...     ctx.quadraticCurveTo(width * 0.76, height * 0.88, width, height * 0.74);
src/render/Ren ...     ctx.lineTo(width, height);
src/render/Ren ...     ctx.lineTo(0, height);
src/render/Ren ...     ctx.closePath();
src/render/Ren ...     ctx.fill();
src/render/Ren ...
src/render/Ren ...     ctx.fillStyle = 'rgba(255, 235, 146, 0.86)';
src/render/Ren ...     for (let i = 0; i < 9; i += 1) {
src/render/Ren ...       const angle = (i / 9) * Math.PI * 2;
src/render/Ren ...       const x = portalX + Math.cos(angle) * portalRadius * (0.72 + (i % 2) * 0.18);
src/render/Ren ...       const y = portalY + Math.sin(angle) * portalRadius * (0.5 + (i % 3) * 0.08);
src/render/Ren ...       this.drawResultSpark(x, y, 5 + (i % 3) * 2);
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
```

```
src/render/Ren ...  drawLoseResultBackground() {
src/render/Ren ...    const ctx = this.ctx;
src/render/Ren ...    const width = this.width;
src/render/Ren ...    const height = this.height;
src/render/Ren ...    ctx.save();
src/render/Ren ...
src/render/Ren ...    const gloom = ctx.createLinearGradient(0, 0, 0, height);
src/render/Ren ...    gloom.addColorStop(0, 'rgba(8, 10, 27, 0.96)');
src/render/Ren ...    gloom.addColorStop(0.5, 'rgba(38, 29, 55, 0.93)');
src/render/Ren ...    gloom.addColorStop(1, 'rgba(95, 31, 45, 0.88)');
src/render/Ren ...    ctx.fillStyle = gloom;
src/render/Ren ...    ctx.fillRect(0, 0, width, height);
src/render/Ren ...
src/render/Ren ...    ctx.strokeStyle = 'rgba(172, 70, 99, 0.28)';
src/render/Ren ...    ctx.lineWidth = 2;
src/render/Ren ...    for (let i = 0; i < 8; i += 1) {
src/render/Ren ...      const y = height * (0.18 + i * 0.085);
src/render/Ren ...      ctx.beginPath();
src/render/Ren ...      ctx.moveTo(width * 0.18, y);
src/render/Ren ...      ctx.quadraticCurveTo(width * 0.5, y + Math.sin(i) * 24, width * 0.82, y - 10);
src/render/Ren ...      ctx.stroke();
src/render/Ren ...    }
src/render/Ren ...
src/render/Ren ...    const riftX = width * 0.5;
src/render/Ren ...    const riftY = height * 0.66;
src/render/Ren ...    const rift = ctx.createRadialGradient(riftX, riftY, 0, riftX, riftY, Math.min(width, height) * 0.34);
src/render/Ren ...    rift.addColorStop(0, 'rgba(255, 99, 91, 0.4)');
src/render/Ren ...    rift.addColorStop(0.45, 'rgba(116, 41, 72, 0.3)');
src/render/Ren ...    rift.addColorStop(1, 'rgba(8, 10, 27, 0)');
src/render/Ren ...    ctx.fillStyle = rift;
src/render/Ren ...    ctx.beginPath();
src/render/Ren ...    ctx.arc(riftX, riftY, Math.min(width, height) * 0.34, 0, Math.PI * 2);
src/render/Ren ...    ctx.fill();
src/render/Ren ...
src/render/Ren ...    ctx.shadowColor = '#ff675f';
src/render/Ren ...    ctx.shadowBlur = 18;
src/render/Ren ...    ctx.strokeStyle = 'rgba(255, 116, 103, 0.78)';
src/render/Ren ...    ctx.lineWidth = 4;
src/render/Ren ...    ctx.beginPath();
src/render/Ren ...    ctx.moveTo(riftX - width * 0.04, riftY - height * 0.19);
src/render/Ren ...    ctx.lineTo(riftX + width * 0.03, riftY - height * 0.08);
src/render/Ren ...    ctx.lineTo(riftX - width * 0.015, riftY + height * 0.02);
src/render/Ren ...    ctx.lineTo(riftX + width * 0.055, riftY + height * 0.17);
src/render/Ren ...    ctx.stroke();
src/render/Ren ...
src/render/Ren ...    ctx.shadowBlur = 0;
src/render/Ren ...    this.drawBrokenResultHeart(riftX, riftY + height * 0.09, Math.min(width, height) * 0.11);
src/render/Ren ...
src/render/Ren ...    ctx.fillStyle = 'rgba(255, 186, 170, 0.42)';
src/render/Ren ...    for (let i = 0; i < 7; i += 1) {
```

```
src/render/Ren ...       const x = width * (0.18 + (i % 4) * 0.21);
src/render/Ren ...       const y = height * (0.72 + Math.floor(i / 4) * 0.08);
src/render/Ren ...       this.drawResultSpark(x, y, 3 + (i % 2));
src/render/Ren ...     }
src/render/Ren ...
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawResultSpark(x, y, size) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.moveTo(x, y - size);
src/render/Ren ...     ctx.lineTo(x + size * 0.34, y - size * 0.34);
src/render/Ren ...     ctx.lineTo(x + size, y);
src/render/Ren ...     ctx.lineTo(x + size * 0.34, y + size * 0.34);
src/render/Ren ...     ctx.lineTo(x, y + size);
src/render/Ren ...     ctx.lineTo(x - size * 0.34, y + size * 0.34);
src/render/Ren ...     ctx.lineTo(x - size, y);
src/render/Ren ...     ctx.lineTo(x - size * 0.34, y - size * 0.34);
src/render/Ren ...     ctx.closePath();
src/render/Ren ...     ctx.fill();
src/render/Ren ...   }
src/render/Ren ...
src/render/Ren ...   drawBrokenResultHeart(x, y, radius) {
src/render/Ren ...     const ctx = this.ctx;
src/render/Ren ...     ctx.save();
src/render/Ren ...     ctx.translate(x, y);
src/render/Ren ...     ctx.shadowColor = '#ff675f';
src/render/Ren ...     ctx.shadowBlur = 16;
src/render/Ren ...     ctx.fillStyle = 'rgba(255, 110, 121, 0.86)';
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.moveTo(0, radius * 0.62);
src/render/Ren ...     ctx.bezierCurveTo(-radius * 1.12, -radius * 0.12, -radius * 0.58, -radius * 1.12, 0, -radius * 0.54);
src/render/Ren ...     ctx.bezierCurveTo(radius * 0.58, -radius * 1.12, radius * 1.12, -radius * 0.12, 0, radius * 0.62);
src/render/Ren ...     ctx.fill();
src/render/Ren ...
src/render/Ren ...     ctx.shadowBlur = 0;
src/render/Ren ...     ctx.strokeStyle = '#3a1730';
src/render/Ren ...     ctx.lineWidth = Math.max(2, radius * 0.12);
src/render/Ren ...     ctx.beginPath();
src/render/Ren ...     ctx.moveTo(-radius * 0.08, -radius * 0.5);
src/render/Ren ...     ctx.lineTo(radius * 0.14, -radius * 0.12);
src/render/Ren ...     ctx.lineTo(-radius * 0.02, radius * 0.08);
src/render/Ren ...     ctx.lineTo(radius * 0.16, radius * 0.46);
src/render/Ren ...     ctx.stroke();
src/render/Ren ...     ctx.restore();
src/render/Ren ...   }
src/render/Ren ... }
src/render/Ren ...
src/render/Ren ... Renderer.getSymbolIndicators = getSymbolIndicators;
```

```
src/render/Ren ... Renderer.getLevelLabel = getLevelLabel;
src/render/Ren ... Renderer.getComboLabel = getComboLabel;
src/render/Ren ... Renderer.getComboTier = getComboTier;
src/render/Ren ... Renderer.getComboPulse = getComboPulse;
src/render/Ren ... Renderer.getSoundToggleLabel = getSoundToggleLabel;
src/render/Ren ... Renderer.getHeartSlotPosition = getHeartSlotPosition;
src/render/Ren ... Renderer.getTransitionCopy = getTransitionCopy;
src/render/Ren ... Renderer.getWinTitle = getWinTitle;
src/render/Ren ... Renderer.getEnemySpriteFrame = getEnemySpriteFrame;
src/render/Ren ... Renderer.ENEMY_SPRITE = ENEMY_SPRITE;
src/render/Ren ... Renderer.SYMBOL_ICON_ASSET_KEYS = SYMBOL_ICON_ASSET_KEYS;
src/render/Ren ... Renderer.getHeroSpriteAnimation = getHeroSpriteAnimation;
src/render/Ren ... Renderer.getHeroSpriteFrame = getHeroSpriteFrame;
src/render/Ren ... Renderer.HERO_SPRITE = HERO_SPRITE;
src/render/Ren ...
src/render/Ren ... module.exports = Renderer;
src/render/Ren ...
src/levels/Level ... // ===== 文件：src/levels/LevelDirector.js =====
src/levels/Level ... 'use strict';
src/levels/Level ...
src/levels/Level ... const { SYMBOLS } = require('../input/GestureRecognizer');
src/levels/Level ... const Enemy = require('../entities/Enemy');
src/levels/Level ... const { THEME_ORDER, getTheme } = require('./Themes');
src/levels/Level ...
src/levels/Level ... const LANES = [
src/levels/Level ...   { x: 0.12, y: 0.2 },
src/levels/Level ...   { x: 0.88, y: 0.2 },
src/levels/Level ...   { x: 0.2, y: 0.08 },
src/levels/Level ...   { x: 0.8, y: 0.08 },
src/levels/Level ...   { x: 0.5, y: -0.03 }
src/levels/Level ... ];
src/levels/Level ...
src/levels/Level ... const CASTLE_TIMELINE = [
src/levels/Level ...   { time: 1.0, lane: 0, symbols: [SYMBOLS.RIGHT], speed: 42 },
src/levels/Level ...   { time: 3.0, lane: 1, symbols: [SYMBOLS.LEFT], speed: 42 },
src/levels/Level ...   { time: 5.2, lane: 2, symbols: [SYMBOLS.DOWN], speed: 46 },
src/levels/Level ...   { time: 7.2, lane: 3, symbols: [SYMBOLS.UP], speed: 46 },
src/levels/Level ...   { time: 10.0, lane: 0, symbols: [SYMBOLS.V], speed: 48 },
src/levels/Level ...   { time: 12.5, lane: 1, symbols: [SYMBOLS.L], speed: 48 },
src/levels/Level ...   { time: 15.0, lane: 4, symbols: [SYMBOLS.LEFT, SYMBOLS.RIGHT], speed: 34, radius: 28, score: 160 },
src/levels/Level ...   { time: 18.0, lane: 0, symbols: [SYMBOLS.UP], speed: 62, radius: 21, score: 120 },
src/levels/Level ...   { time: 20.2, lane: 1, symbols: [SYMBOLS.DOWN], speed: 62, radius: 21, score: 120 },
src/levels/Level ...   { time: 23.0, lane: 2, symbols: [SYMBOLS.V, SYMBOLS.UP], speed: 38, radius: 29, score: 180 },
src/levels/Level ...   { time: 26.0, lane: 3, symbols: [SYMBOLS.L, SYMBOLS.DOWN], speed: 38, radius: 29, score: 180 },
src/levels/Level ...   { time: 29.5, lane: 0, symbols: [SYMBOLS.RIGHT, SYMBOLS.RIGHT], speed: 46, score: 170 },
src/levels/Level ...   { time: 32.0, lane: 1, symbols: [SYMBOLS.LEFT, SYMBOLS.LEFT], speed: 46, score: 170 },
src/levels/Level ...   { time: 35.0, lane: 4, symbols: [SYMBOLS.DOWN, SYMBOLS.V, SYMBOLS.UP], speed: 30, radius: 32, score: 240 },
src/levels/Level ...   { time: 39.5, lane: 0, symbols: [SYMBOLS.UP], speed: 72, radius: 20, score: 130 },
src/levels/Level ...   { time: 41.0, lane: 1, symbols: [SYMBOLS.DOWN], speed: 72, radius: 20, score: 130 },
src/levels/Level ...   { time: 43.0, lane: 2, symbols: [SYMBOLS.LEFT, SYMBOLS.V], speed: 46, radius: 27, score: 190 },
```

```
src/levels/Level ...   { time: 46.0, lane: 3, symbols: [SYMBOLS.RIGHT, SYMBOLS.L], speed: 46, radius: 27, score: 190 },
src/levels/Level ...   { time: 50.5, lane: 4, symbols: [SYMBOLS.UP, SYMBOLS.DOWN, SYMBOLS.V], speed: 28, radius: 35, score: 260 },
src/levels/Level ...   { time: 56.0, lane: 0, symbols: [SYMBOLS.RIGHT], speed: 78, radius: 20, score: 140 },
src/levels/Level ...   { time: 57.2, lane: 1, symbols: [SYMBOLS.LEFT], speed: 78, radius: 20, score: 140 },
src/levels/Level ...   { time: 60.0, lane: 4, symbols: [SYMBOLS.LEFT, SYMBOLS.RIGHT, SYMBOLS.L, SYMBOLS.LEFT], speed: 26, radius: 42, score: 500, ki ...
src/levels/Level ...  ];
src/levels/Level ...
src/levels/Level ...  const OCEAN_TIMELINE = [
src/levels/Level ...   { time: 1.0, lane: 0, symbols: [SYMBOLS.UP], speed: 48 },
src/levels/Level ...   { time: 2.8, lane: 1, symbols: [SYMBOLS.RIGHT], speed: 48 },
src/levels/Level ...   { time: 5.0, lane: 2, symbols: [SYMBOLS.V], speed: 52 },
src/levels/Level ...   { time: 7.0, lane: 3, symbols: [SYMBOLS.LEFT, SYMBOLS.DOWN], speed: 48, radius: 27, score: 170 },
src/levels/Level ...   { time: 10.0, lane: 0, symbols: [SYMBOLS.L, SYMBOLS.RIGHT], speed: 48, radius: 28, score: 180 },
src/levels/Level ...   { time: 12.2, lane: 1, symbols: [SYMBOLS.DOWN], speed: 58, radius: 22, score: 130 },
src/levels/Level ...   { time: 14.6, lane: 4, symbols: [SYMBOLS.RIGHT, SYMBOLS.V], speed: 45, radius: 29, score: 190 },
src/levels/Level ...   { time: 17.4, lane: 2, symbols: [SYMBOLS.LEFT, SYMBOLS.UP], speed: 47, radius: 29, score: 190 },
src/levels/Level ...   { time: 20.0, lane: 3, symbols: [SYMBOLS.UP], speed: 68, radius: 21, score: 140 },
src/levels/Level ...   { time: 22.0, lane: 0, symbols: [SYMBOLS.V, SYMBOLS.DOWN, SYMBOLS.RIGHT], speed: 40, radius: 32, score: 260 },
src/levels/Level ...   { time: 25.2, lane: 1, symbols: [SYMBOLS.RIGHT, SYMBOLS.RIGHT], speed: 58, radius: 30, score: 210 },
src/levels/Level ...   { time: 27.5, lane: 2, symbols: [SYMBOLS.LEFT, SYMBOLS.LEFT], speed: 58, radius: 30, score: 210 },
src/levels/Level ...   { time: 30.2, lane: 3, symbols: [SYMBOLS.L, SYMBOLS.V], speed: 54, radius: 29, score: 210 },
src/levels/Level ...   { time: 33.0, lane: 4, symbols: [SYMBOLS.DOWN, SYMBOLS.UP, SYMBOLS.RIGHT], speed: 43, radius: 34, score: 290 },
src/levels/Level ...   { time: 37.0, lane: 0, symbols: [SYMBOLS.RIGHT], speed: 84, radius: 20, score: 150 },
src/levels/Level ...   { time: 38.4, lane: 1, symbols: [SYMBOLS.LEFT], speed: 84, radius: 20, score: 150 },
src/levels/Level ...   { time: 41.0, lane: 2, symbols: [SYMBOLS.V, SYMBOLS.UP], speed: 62, radius: 30, score: 230 },
src/levels/Level ...   { time: 43.2, lane: 3, symbols: [SYMBOLS.L, SYMBOLS.DOWN], speed: 62, radius: 30, score: 230 },
src/levels/Level ...   { time: 46.8, lane: 4, symbols: [SYMBOLS.UP, SYMBOLS.DOWN, SYMBOLS.V], speed: 44, radius: 34, score: 300 },
src/levels/Level ...   { time: 51.0, lane: 0, symbols: [SYMBOLS.LEFT, SYMBOLS.V, SYMBOLS.RIGHT], speed: 55, radius: 33, score: 320 },
src/levels/Level ...   { time: 54.5, lane: 1, symbols: [SYMBOLS.RIGHT, SYMBOLS.L], speed: 67, radius: 28, score: 230 },
src/levels/Level ...   { time: 60.0, lane: 4, symbols: [SYMBOLS.LEFT, SYMBOLS.RIGHT, SYMBOLS.V, SYMBOLS.L, SYMBOLS.LEFT], speed: 32, radius: 48, ...
src/levels/Level ...  ];
src/levels/Level ...
src/levels/Level ...  const PENGUIN_HOTEL_TIMELINE = [
src/levels/Level ...   { time: 0.9, lane: 0, symbols: [SYMBOLS.CIRCLE, SYMBOLS.RIGHT], speed: 55, score: 180 },
src/levels/Level ...   { time: 2.4, lane: 1, symbols: [SYMBOLS.CIRCLE, SYMBOLS.UP], speed: 55, score: 180 },
src/levels/Level ...   { time: 4.3, lane: 2, symbols: [SYMBOLS.CIRCLE, SYMBOLS.V], speed: 60, score: 190 },
src/levels/Level ...   { time: 6.0, lane: 3, symbols: [SYMBOLS.LEFT, SYMBOLS.CIRCLE], speed: 55, radius: 27, score: 210 },
src/levels/Level ...   { time: 8.5, lane: 0, symbols: [SYMBOLS.L, SYMBOLS.CIRCLE], speed: 55, radius: 28, score: 220 },
src/levels/Level ...   { time: 10.4, lane: 1, symbols: [SYMBOLS.DOWN, SYMBOLS.CIRCLE], speed: 67, radius: 23, score: 200 },
src/levels/Level ...   { time: 12.4, lane: 4, symbols: [SYMBOLS.RIGHT, SYMBOLS.CIRCLE, SYMBOLS.V], speed: 52, radius: 30, score: 280 },
src/levels/Level ...   { time: 14.8, lane: 2, symbols: [SYMBOLS.LEFT, SYMBOLS.CIRCLE, SYMBOLS.UP], speed: 54, radius: 30, score: 280 },
src/levels/Level ...   { time: 17.0, lane: 3, symbols: [SYMBOLS.UP, SYMBOLS.CIRCLE], speed: 78, radius: 22, score: 220 },
src/levels/Level ...   { time: 18.7, lane: 0, symbols: [SYMBOLS.V, SYMBOLS.DOWN, SYMBOLS.CIRCLE], speed: 46, radius: 33, score: 310 },
src/levels/Level ...   { time: 21.4, lane: 1, symbols: [SYMBOLS.CIRCLE, SYMBOLS.RIGHT, SYMBOLS.CIRCLE], speed: 67, radius: 31, score: 290 },
src/levels/Level ...   { time: 23.4, lane: 2, symbols: [SYMBOLS.CIRCLE, SYMBOLS.LEFT, SYMBOLS.CIRCLE], speed: 67, radius: 31, score: 290 },
src/levels/Level ...   { time: 25.7, lane: 3, symbols: [SYMBOLS.L, SYMBOLS.V, SYMBOLS.CIRCLE], speed: 62, radius: 30, score: 300 },
src/levels/Level ...   { time: 28.1, lane: 4, symbols: [SYMBOLS.DOWN, SYMBOLS.CIRCLE, SYMBOLS.UP, SYMBOLS.RIGHT], speed: 49, radius: 35, score:  ...
src/levels/Level ...   { time: 31.5, lane: 0, symbols: [SYMBOLS.RIGHT, SYMBOLS.CIRCLE], speed: 97, radius: 21, score: 240 },
src/levels/Level ...   { time: 32.6, lane: 1, symbols: [SYMBOLS.LEFT, SYMBOLS.CIRCLE], speed: 97, radius: 21, score: 240 },
src/levels/Level ...   { time: 34.9, lane: 2, symbols: [SYMBOLS.V, SYMBOLS.CIRCLE, SYMBOLS.UP], speed: 71, radius: 31, score: 330 },
```

```
src/levels/Level ...   { time: 36.7, lane: 3, symbols: [SYMBOLS.CIRCLE, SYMBOLS.L, SYMBOLS.DOWN], speed: 71, radius: 31, score: 330 },
src/levels/Level ...   { time: 39.8, lane: 4, symbols: [SYMBOLS.UP, SYMBOLS.CIRCLE, SYMBOLS.DOWN, SYMBOLS.V], speed: 51, radius: 35, score: 400 },
src/levels/Level ...   { time: 43.4, lane: 0, symbols: [SYMBOLS.LEFT, SYMBOLS.CIRCLE, SYMBOLS.V, SYMBOLS.RIGHT], speed: 63, radius: 34, score: 430 },
src/levels/Level ...   { time: 46.3, lane: 1, symbols: [SYMBOLS.RIGHT, SYMBOLS.CIRCLE, SYMBOLS.L], speed: 77, radius: 29, score: 340 },
src/levels/Level ...   { time: 51.0, lane: 4, symbols: [SYMBOLS.CIRCLE, SYMBOLS.CIRCLE, SYMBOLS.LEFT, SYMBOLS.RIGHT, SYMBOLS.V, SYMBOLS.L ...
src/levels/Level ...  ];
src/levels/Level ...
src/levels/Level ...  const DINOSAUR_PARK_TIMELINE = [
src/levels/Level ...   { time: 0.9, lane: 0, symbols: [SYMBOLS.CIRCLE, SYMBOLS.RIGHT], speed: 59, score: 220 },
src/levels/Level ...   { time: 2.4, lane: 1, symbols: [SYMBOLS.CIRCLE, SYMBOLS.UP], speed: 59, score: 220 },
src/levels/Level ...   { time: 4.3, lane: 2, symbols: [SYMBOLS.CIRCLE, SYMBOLS.V], speed: 64, score: 230 },
src/levels/Level ...   { time: 6.0, lane: 3, symbols: [SYMBOLS.LEFT, SYMBOLS.Z], speed: 59, radius: 27, score: 250 },
src/levels/Level ...   { time: 8.5, lane: 0, symbols: [SYMBOLS.L, SYMBOLS.CIRCLE], speed: 59, radius: 28, score: 260 },
src/levels/Level ...   { time: 10.4, lane: 1, symbols: [SYMBOLS.DOWN, SYMBOLS.Z], speed: 71, radius: 23, score: 240 },
src/levels/Level ...   { time: 12.4, lane: 4, symbols: [SYMBOLS.RIGHT, SYMBOLS.CIRCLE, SYMBOLS.V], speed: 56, radius: 30, score: 320 },
src/levels/Level ...   { time: 14.8, lane: 2, symbols: [SYMBOLS.LEFT, SYMBOLS.Z, SYMBOLS.UP], speed: 58, radius: 30, score: 320 },
src/levels/Level ...   { time: 17.0, lane: 3, symbols: [SYMBOLS.UP, SYMBOLS.CIRCLE], speed: 82, radius: 22, score: 260 },
src/levels/Level ...   { time: 18.7, lane: 0, symbols: [SYMBOLS.V, SYMBOLS.DOWN, SYMBOLS.Z], speed: 50, radius: 33, score: 350 },
src/levels/Level ...   { time: 21.4, lane: 1, symbols: [SYMBOLS.CIRCLE, SYMBOLS.RIGHT, SYMBOLS.CIRCLE], speed: 71, radius: 31, score: 330 },
src/levels/Level ...   { time: 23.4, lane: 2, symbols: [SYMBOLS.Z, SYMBOLS.LEFT, SYMBOLS.CIRCLE], speed: 71, radius: 31, score: 330 },
src/levels/Level ...   { time: 25.7, lane: 3, symbols: [SYMBOLS.L, SYMBOLS.V, SYMBOLS.CIRCLE], speed: 66, radius: 30, score: 340 },
src/levels/Level ...   { time: 28.1, lane: 4, symbols: [SYMBOLS.DOWN, SYMBOLS.Z, SYMBOLS.UP, SYMBOLS.RIGHT], speed: 53, radius: 35, score: 420 },
src/levels/Level ...   { time: 31.5, lane: 0, symbols: [SYMBOLS.RIGHT, SYMBOLS.CIRCLE], speed: 101, radius: 21, score: 280 },
src/levels/Level ...   { time: 32.6, lane: 1, symbols: [SYMBOLS.LEFT, SYMBOLS.Z], speed: 101, radius: 21, score: 280 },
src/levels/Level ...   { time: 34.9, lane: 2, symbols: [SYMBOLS.V, SYMBOLS.CIRCLE, SYMBOLS.UP], speed: 75, radius: 31, score: 370 },
src/levels/Level ...   { time: 36.7, lane: 3, symbols: [SYMBOLS.Z, SYMBOLS.L, SYMBOLS.DOWN], speed: 75, radius: 31, score: 370 },
src/levels/Level ...   { time: 39.8, lane: 4, symbols: [SYMBOLS.UP, SYMBOLS.CIRCLE, SYMBOLS.DOWN, SYMBOLS.V], speed: 55, radius: 35, score: 440 },
src/levels/Level ...   { time: 43.4, lane: 0, symbols: [SYMBOLS.LEFT, SYMBOLS.Z, SYMBOLS.V, SYMBOLS.RIGHT], speed: 67, radius: 34, score: 470 },
src/levels/Level ...   { time: 46.3, lane: 1, symbols: [SYMBOLS.RIGHT, SYMBOLS.CIRCLE, SYMBOLS.L], speed: 81, radius: 29, score: 380 },
src/levels/Level ...   { time: 51.0, lane: 4, symbols: [SYMBOLS.Z, SYMBOLS.CIRCLE, SYMBOLS.RIGHT, SYMBOLS.LEFT, SYMBOLS.V, SYMBOLS.L, SYM ...
src/levels/Level ...  ];
src/levels/Level ...
src/levels/Level ...  const SKY_CITY_TIMELINE = DINOSAUR_PARK_TIMELINE.slice(0, -1).map((spawn, index) => ({
src/levels/Level ...    ...spawn,
src/levels/Level ...    speed: spawn.speed + 4,
src/levels/Level ...    score: (spawn.score || 100) + 40,
src/levels/Level ...    symbols: spawn.symbols.map((symbol, symbolIndex) => (
src/levels/Level ...      symbol === SYMBOLS.Z && (index + symbolIndex) % 2 === 0 ? SYMBOLS.M : symbol
src/levels/Level ...    ))
src/levels/Level ...  })).concat([{
src/levels/Level ...    time: 51.0,
src/levels/Level ...    lane: 4,
src/levels/Level ...    symbols: [
src/levels/Level ...      SYMBOLS.M, SYMBOLS.Z, SYMBOLS.CIRCLE, SYMBOLS.RIGHT,
src/levels/Level ...      SYMBOLS.LEFT, SYMBOLS.V, SYMBOLS.L, SYMBOLS.M
src/levels/Level ...    ],
src/levels/Level ...    speed: 45,
src/levels/Level ...    radius: 58,
src/levels/Level ...    score: 1600,
src/levels/Level ...    kind: 'boss'
```

```
src/levels/Level ... }]);
src/levels/Level ...
src/levels/Level ... const SEA_TRAIN_TIMELINE = SKY_CITY_TIMELINE.slice(0, -1).map((spawn, index) => ({
src/levels/Level ...   ...spawn,
src/levels/Level ...   speed: spawn.speed + 3,
src/levels/Level ...   score: (spawn.score || 100) + 50,
src/levels/Level ...   symbols: spawn.symbols.map((symbol, symbolIndex) => {
src/levels/Level ...     if (symbol === SYMBOLS.M && (index + symbolIndex) % 2 === 0) {
src/levels/Level ...       return SYMBOLS.S;
src/levels/Level ...     }
src/levels/Level ...     if (symbol === SYMBOLS.Z && (index + symbolIndex) % 3 === 0) {
src/levels/Level ...       return SYMBOLS.S;
src/levels/Level ...     }
src/levels/Level ...     return symbol;
src/levels/Level ...   })
src/levels/Level ... })).concat([{
src/levels/Level ...   time: 51.0,
src/levels/Level ...   lane: 4,
src/levels/Level ...   symbols: [
src/levels/Level ...     SYMBOLS.S, SYMBOLS.M, SYMBOLS.Z, SYMBOLS.CIRCLE, SYMBOLS.RIGHT,
src/levels/Level ...     SYMBOLS.LEFT, SYMBOLS.V, SYMBOLS.L, SYMBOLS.S
src/levels/Level ...   ],
src/levels/Level ...   speed: 49,
src/levels/Level ...   radius: 62,
src/levels/Level ...   score: 1900,
src/levels/Level ...   kind: 'boss'
src/levels/Level ... }]);
src/levels/Level ...
src/levels/Level ... const LEVEL_TIME_REDUCTION = 15;
src/levels/Level ...
src/levels/Level ... function reduceTimelineDuration(timeline, reduction) {
src/levels/Level ...   const firstTime = timeline[0].time;
src/levels/Level ...   const lastTime = timeline[timeline.length - 1].time;
src/levels/Level ...   const targetLastTime = Math.max(firstTime, lastTime - reduction);
src/levels/Level ...   const originalSpan = lastTime - firstTime;
src/levels/Level ...   const targetSpan = targetLastTime - firstTime;
src/levels/Level ...
src/levels/Level ...   if (originalSpan <= 0) {
src/levels/Level ...     return timeline.map((spawn) => ({ ...spawn, time: targetLastTime }));
src/levels/Level ...   }
src/levels/Level ...
src/levels/Level ...   return timeline.map((spawn) => ({
src/levels/Level ...     ...spawn,
src/levels/Level ...     time: Number((firstTime + ((spawn.time - firstTime) * targetSpan / originalSpan)).toFixed(2))
src/levels/Level ...   }));
src/levels/Level ... }
src/levels/Level ...
src/levels/Level ... const LEVELS = [
src/levels/Level ...   {
src/levels/Level ...     symbolDisplay: 'queue',
```

```
src/levels/Level ...     timeline: reduceTimelineDuration(CASTLE_TIMELINE, LEVEL_TIME_REDUCTION)
src/levels/Level ...   },
src/levels/Level ...   {
src/levels/Level ...     symbolDisplay: 'current-and-dots',
src/levels/Level ...     timeline: reduceTimelineDuration(OCEAN_TIMELINE, LEVEL_TIME_REDUCTION)
src/levels/Level ...   },
src/levels/Level ...   {
src/levels/Level ...     symbolDisplay: 'current-and-dots',
src/levels/Level ...     timeline: reduceTimelineDuration(PENGUIN_HOTEL_TIMELINE, LEVEL_TIME_REDUCTION)
src/levels/Level ...   },
src/levels/Level ...   {
src/levels/Level ...     symbolDisplay: 'current-and-dots',
src/levels/Level ...     timeline: reduceTimelineDuration(DINOSAUR_PARK_TIMELINE, LEVEL_TIME_REDUCTION)
src/levels/Level ...   },
src/levels/Level ...   {
src/levels/Level ...     symbolDisplay: 'current-and-dots',
src/levels/Level ...     timeline: reduceTimelineDuration(SKY_CITY_TIMELINE, LEVEL_TIME_REDUCTION)
src/levels/Level ...   },
src/levels/Level ...   {
src/levels/Level ...     symbolDisplay: 'current-and-dots',
src/levels/Level ...     timeline: reduceTimelineDuration(SEA_TRAIN_TIMELINE, LEVEL_TIME_REDUCTION)
src/levels/Level ...   }
src/levels/Level ... ];
src/levels/Level ...
src/levels/Level ... class LevelDirector {
src/levels/Level ...   constructor(width, height) {
src/levels/Level ...     this.width = width;
src/levels/Level ...     this.height = height;
src/levels/Level ...     this.levelCount = LEVELS.length;
src/levels/Level ...     this.startLevel(1);
src/levels/Level ...   }
src/levels/Level ...
src/levels/Level ...   reset(width, height, level, themeId) {
src/levels/Level ...     this.width = width || this.width;
src/levels/Level ...     this.height = height || this.height;
src/levels/Level ...     this.startLevel(level || 1, themeId);
src/levels/Level ...   }
src/levels/Level ...
src/levels/Level ...   startLevel(level, themeId) {
src/levels/Level ...     const selectedLevel = Math.max(1, Math.min(this.levelCount, level || 1));
src/levels/Level ...     this.level = selectedLevel;
src/levels/Level ...     this.config = LEVELS[selectedLevel - 1];
src/levels/Level ...     this.theme = getTheme(themeId || THEME_ORDER[selectedLevel - 1]);
src/levels/Level ...     this.themeId = this.theme.id;
src/levels/Level ...     this.index = 0;
src/levels/Level ...     this.total = this.config.timeline.length;
src/levels/Level ...   }
src/levels/Level ...
src/levels/Level ...   update(elapsed, enemies) {
src/levels/Level ...     const timeline = this.config.timeline;
```

```
src/levels/Level ...     while (this.index < timeline.length && elapsed >= timeline[this.index].time) {
src/levels/Level ...       enemies.push(this.createEnemy(timeline[this.index], this.index));
src/levels/Level ...       this.index += 1;
src/levels/Level ...     }
src/levels/Level ...   }
src/levels/Level ...
src/levels/Level ...   createEnemy(spawn, spawnIndex) {
src/levels/Level ...     const lane = LANES[spawn.lane % LANES.length];
src/levels/Level ...     return new Enemy({
src/levels/Level ...       x: lane.x * this.width,
src/levels/Level ...       y: lane.y * this.height,
src/levels/Level ...       speed: spawn.speed,
src/levels/Level ...       symbols: spawn.symbols,
src/levels/Level ...       radius: spawn.radius,
src/levels/Level ...       score: spawn.score,
src/levels/Level ...       kind: spawn.kind,
src/levels/Level ...       species: this.theme.species[spawnIndex] || this.theme.species[0],
src/levels/Level ...       symbolDisplay: this.config.symbolDisplay
src/levels/Level ...     });
src/levels/Level ...   }
src/levels/Level ...
src/levels/Level ...   isComplete(enemies) {
src/levels/Level ...     return this.index >= this.config.timeline.length && enemies.length === 0;
src/levels/Level ...   }
src/levels/Level ... }
src/levels/Level ...
src/levels/Level ... LevelDirector.LEVELS = LEVELS;
src/levels/Level ... LevelDirector.LEVEL_TIME_REDUCTION = LEVEL_TIME_REDUCTION;
src/levels/Level ...
src/levels/Level ... module.exports = LevelDirector;
src/levels/Level ...
 src/levels/The ... // ===== 文件：src/levels/Themes.js =====
 src/levels/The ... 'use strict';
 src/levels/The ...
 src/levels/The ... const THEME_IDS = {
 src/levels/The ...   CASTLE: 'castle',
 src/levels/The ...   OCEAN: 'ocean',
 src/levels/The ...   PENGUIN_HOTEL: 'penguinHotel',
 src/levels/The ...   DINOSAUR_PARK: 'dinosaurPark',
 src/levels/The ...   SKY_CITY: 'skyCity',
 src/levels/The ...   SEA_TRAIN: 'seaTrain'
 src/levels/The ... };
 src/levels/The ...
 src/levels/The ... const THEME_ORDER = [
 src/levels/The ...   THEME_IDS.CASTLE,
 src/levels/The ...   THEME_IDS.OCEAN,
 src/levels/The ...   THEME_IDS.PENGUIN_HOTEL,
 src/levels/The ...   THEME_IDS.DINOSAUR_PARK,
 src/levels/The ...   THEME_IDS.SKY_CITY,
 src/levels/The ...   THEME_IDS.SEA_TRAIN
```

```
src/levels/The ... ];
src/levels/The ...
src/levels/The ... const THEMES = {
src/levels/The ...   [THEME_IDS.CASTLE]: {
src/levels/The ...     id: THEME_IDS.CASTLE,
src/levels/The ...     name: '幽光古堡',
src/levels/The ...     backgroundAsset: 'castleCorridorLoop',
src/levels/The ...     music: 'assets/audio/bgm-castle.wav',
src/levels/The ...     runeColor: 'rgba(255, 235, 171, 0.18)',
src/levels/The ...     species: new Array(22).fill('ghost')
src/levels/The ...   },
src/levels/The ...   [THEME_IDS.OCEAN]: {
src/levels/The ...     id: THEME_IDS.OCEAN,
src/levels/The ...     name: '深海飞船长廊',
src/levels/The ...     backgroundAsset: 'oceanSpaceshipCorridorLoop',
src/levels/The ...     music: 'assets/audio/bgm-ocean.wav',
src/levels/The ...     runeColor: 'rgba(115, 244, 255, 0.22)',
src/levels/The ...     species: [
src/levels/The ...       'jellyfish', 'jellyfish', 'jellyfish', 'jellyfish', 'pufferfish', 'jellyfish',
src/levels/The ...       'pufferfish', 'pufferfish', 'jellyfish', 'pufferfish', 'shark', 'shark',
src/levels/The ...       'pufferfish', 'shark', 'jellyfish', 'jellyfish', 'shark', 'shark',
src/levels/The ...       'pufferfish', 'shark', 'pufferfish', 'megalodon'
src/levels/The ...     ]
src/levels/The ...   },
src/levels/The ...   [THEME_IDS.PENGUIN_HOTEL]: {
src/levels/The ...     id: THEME_IDS.PENGUIN_HOTEL,
src/levels/The ...     name: '极光企鹅酒店',
src/levels/The ...     backgroundAsset: 'penguinHotelCorridorLoop',
src/levels/The ...     music: 'assets/audio/bgm-penguin-hotel.wav',
src/levels/The ...     runeColor: 'rgba(172, 249, 255, 0.28)',
src/levels/The ...     species: [
src/levels/The ...       'penguinBellhop', 'penguinChef', 'penguinBellhop', 'penguinChef',
src/levels/The ...       'penguinBellhop', 'penguinChef', 'penguinBellhop', 'penguinChef',
src/levels/The ...       'penguinBellhop', 'penguinChef', 'penguinBellhop', 'penguinChef',
src/levels/The ...       'penguinBellhop', 'penguinChef', 'penguinBellhop', 'penguinChef',
src/levels/The ...       'penguinBellhop', 'penguinChef', 'penguinBellhop', 'penguinChef',
src/levels/The ...       'penguinBellhop', 'emperorPenguin'
src/levels/The ...     ]
src/levels/The ...   },
src/levels/The ...   [THEME_IDS.DINOSAUR_PARK]: {
src/levels/The ...     id: THEME_IDS.DINOSAUR_PARK,
src/levels/The ...     name: '恐龙乐园',
src/levels/The ...     backgroundAsset: 'dinosaurParkCorridorLoop',
src/levels/The ...     music: 'assets/audio/bgm-dinosaur-park.wav',
src/levels/The ...     runeColor: 'rgba(228, 190, 72, 0.31)',
src/levels/The ...     species: [
src/levels/The ...       'pterosaur', 'triceratops', 'brachiosaurus', 'pterosaur', 'triceratops',
src/levels/The ...       'brachiosaurus', 'pterosaur', 'triceratops', 'brachiosaurus', 'pterosaur',
src/levels/The ...       'triceratops', 'brachiosaurus', 'pterosaur', 'triceratops', 'brachiosaurus',
src/levels/The ...       'pterosaur', 'triceratops', 'brachiosaurus', 'pterosaur', 'triceratops',
```

```
src/levels/The ...       'brachiosaurus', 'tyrannosaurus'
src/levels/The ...     ]
src/levels/The ...   },
src/levels/The ...   [THEME_IDS.SKY_CITY]: {
src/levels/The ...     id: THEME_IDS.SKY_CITY,
src/levels/The ...     name: '天空之城',
src/levels/The ...     backgroundAsset: 'skyCityCorridorLoop',
src/levels/The ...     music: 'assets/audio/bgm-sky-city.wav',
src/levels/The ...     runeColor: 'rgba(255, 220, 132, 0.3)',
src/levels/The ...     species: [
src/levels/The ...       'cloudWisp', 'wingedSentinel', 'cloudWisp', 'wingedSentinel',
src/levels/The ...       'cloudWisp', 'wingedSentinel', 'cloudWisp', 'wingedSentinel',
src/levels/The ...       'cloudWisp', 'wingedSentinel', 'cloudWisp', 'wingedSentinel',
src/levels/The ...       'cloudWisp', 'wingedSentinel', 'cloudWisp', 'wingedSentinel',
src/levels/The ...       'cloudWisp', 'wingedSentinel', 'cloudWisp', 'wingedSentinel',
src/levels/The ...       'cloudWisp', 'templeGriffin'
src/levels/The ...     ]
src/levels/The ...   },
src/levels/The ...   [THEME_IDS.SEA_TRAIN]: {
src/levels/The ...     id: THEME_IDS.SEA_TRAIN,
src/levels/The ...     name: '海上列车',
src/levels/The ...     backgroundAsset: 'seaTrainHorizon',
src/levels/The ...     music: 'assets/audio/bgm-sea-train.wav',
src/levels/The ...     runeColor: 'rgba(255, 198, 132, 0.28)',
src/levels/The ...     species: [
src/levels/The ...       'paperSpirit', 'waveLantern', 'paperSpirit', 'waveLantern',
src/levels/The ...       'paperSpirit', 'waveLantern', 'paperSpirit', 'waveLantern',
src/levels/The ...       'paperSpirit', 'waveLantern', 'paperSpirit', 'waveLantern',
src/levels/The ...       'paperSpirit', 'waveLantern', 'paperSpirit', 'waveLantern',
src/levels/The ...       'paperSpirit', 'waveLantern', 'paperSpirit', 'waveLantern',
src/levels/The ...       'paperSpirit', 'trainConductor'
src/levels/The ...     ]
src/levels/The ...   }
src/levels/The ... };
src/levels/The ...
src/levels/The ... function getTheme(themeId) {
src/levels/The ...   return THEMES[themeId] || THEMES[THEME_IDS.CASTLE];
src/levels/The ... }
src/levels/The ...
src/levels/The ... function createThemeOrder(random) {
src/levels/The ...   const randomValue = random || Math.random;
src/levels/The ...   const order = THEME_ORDER.slice();
src/levels/The ...   for (let i = order.length - 1; i > 0; i -= 1) {
src/levels/The ...     const value = Math.max(0, Math.min(0.9999999999999999, randomValue()));
src/levels/The ...     const selectedIndex = Math.floor(value * (i + 1));
src/levels/The ...     const selectedTheme = order[selectedIndex];
src/levels/The ...     order[selectedIndex] = order[i];
src/levels/The ...     order[i] = selectedTheme;
src/levels/The ...   }
src/levels/The ...   return order;
```

```
src/levels/The ... }
src/levels/The ...
src/levels/The ... module.exports = {
src/levels/The ...   THEME_IDS,
src/levels/The ...   THEME_ORDER,
src/levels/The ...   THEMES,
src/levels/The ...   getTheme,
src/levels/The ...   createThemeOrder
src/levels/The ... };
src/levels/The ...
src/entities/Ene ... // ===== 文件：src/entities/Enemy.js =====
src/entities/Ene ... 'use strict';
src/entities/Ene ...
src/entities/Ene ... const { distance, normalizeVector } = require('../utils/math');
src/entities/Ene ...
src/entities/Ene ... class Enemy {
src/entities/Ene ...   constructor(config) {
src/entities/Ene ...     this.x = config.x;
src/entities/Ene ...     this.y = config.y;
src/entities/Ene ...     this.radius = config.radius || 24;
src/entities/Ene ...     this.speed = config.speed || 42;
src/entities/Ene ...     this.symbols = config.symbols.slice();
src/entities/Ene ...     this.score = config.score === undefined ? 100 : config.score;
src/entities/Ene ...     this.kind = config.kind || 'normal';
src/entities/Ene ...     this.species = config.species || 'ghost';
src/entities/Ene ...     this.symbolDisplay = config.symbolDisplay || 'queue';
src/entities/Ene ...     this.maxSymbols = this.symbols.length;
src/entities/Ene ...     this.hitFlash = 0;
src/entities/Ene ...     this.dead = false;
src/entities/Ene ...     this.reachedHero = false;
src/entities/Ene ...     this.phase = Math.random() * Math.PI * 2;
src/entities/Ene ...   }
src/entities/Ene ...
src/entities/Ene ...   update(dt, hero, movementScale) {
src/entities/Ene ...     if (this.dead) {
src/entities/Ene ...       return;
src/entities/Ene ...     }
src/entities/Ene ...
src/entities/Ene ...     this.phase += dt * 4;
src/entities/Ene ...     this.hitFlash = Math.max(0, this.hitFlash - dt);
src/entities/Ene ...     const travelDt = dt * (movementScale || 1);
src/entities/Ene ...     const dir = normalizeVector(hero.x - this.x, hero.y - this.y);
src/entities/Ene ...     this.x += dir.x * this.speed * travelDt;
src/entities/Ene ...     this.y += dir.y * this.speed * travelDt;
src/entities/Ene ...
src/entities/Ene ...     if (distance(this, hero) < this.radius + hero.radius * 0.72) {
src/entities/Ene ...       this.reachedHero = true;
src/entities/Ene ...       this.dead = true;
src/entities/Ene ...     }
src/entities/Ene ...   }
```

```
src/entities/Ene ...
src/entities/Ene ...   matches(symbol) {
src/entities/Ene ...     return this.symbols[0] === symbol;
src/entities/Ene ...   }
src/entities/Ene ...
src/entities/Ene ...   applySymbol(symbol) {
src/entities/Ene ...     if (!this.matches(symbol)) {
src/entities/Ene ...       return false;
src/entities/Ene ...     }
src/entities/Ene ...
src/entities/Ene ...     this.symbols.shift();
src/entities/Ene ...     this.hitFlash = 0.18;
src/entities/Ene ...     if (this.symbols.length === 0) {
src/entities/Ene ...       this.dead = true;
src/entities/Ene ...     }
src/entities/Ene ...     return true;
src/entities/Ene ...   }
src/entities/Ene ...
src/entities/Ene ...   takeComboHit() {
src/entities/Ene ...     if (this.dead || this.symbols.length === 0) {
src/entities/Ene ...       return false;
src/entities/Ene ...     }
src/entities/Ene ...
src/entities/Ene ...     this.symbols.shift();
src/entities/Ene ...     this.hitFlash = 0.18;
src/entities/Ene ...     if (this.symbols.length === 0) {
src/entities/Ene ...       this.dead = true;
src/entities/Ene ...     }
src/entities/Ene ...     return true;
src/entities/Ene ...   }
src/entities/Ene ... }
src/entities/Ene ...
src/entities/Ene ... module.exports = Enemy;
src/entities/Ene ...
 src/assets/Ass ... // ===== 文件：src/assets/AssetManager.js =====
 src/assets/Ass ... 'use strict';
 src/assets/Ass ...
 src/assets/Ass ... class AssetManager {
 src/assets/Ass ...   constructor(wxApi) {
 src/assets/Ass ...     this.wx = wxApi;
 src/assets/Ass ...     this.images = {};
 src/assets/Ass ...     this.status = {};
 src/assets/Ass ...   }
 src/assets/Ass ...
 src/assets/Ass ...   loadImage(key, src) {
 src/assets/Ass ...     const image = this.createImage();
 src/assets/Ass ...     this.status[key] = 'loading';
 src/assets/Ass ...
 src/assets/Ass ...     image.onload = () => {
 src/assets/Ass ...       this.images[key] = image;
```

```
src/assets/Ass ...       this.status[key] = 'loaded';
src/assets/Ass ...     };
src/assets/Ass ...
src/assets/Ass ...     image.onerror = () => {
src/assets/Ass ...       this.status[key] = 'error';
src/assets/Ass ...     };
src/assets/Ass ...
src/assets/Ass ...     image.src = src;
src/assets/Ass ...     return image;
src/assets/Ass ...   }
src/assets/Ass ...
src/assets/Ass ...   createImage() {
src/assets/Ass ...     if (this.wx && this.wx.createImage) {
src/assets/Ass ...       return this.wx.createImage();
src/assets/Ass ...     }
src/assets/Ass ...
src/assets/Ass ...     if (typeof Image !== 'undefined') {
src/assets/Ass ...       return new Image();
src/assets/Ass ...     }
src/assets/Ass ...
src/assets/Ass ...     return {
src/assets/Ass ...       set src(_src) {
src/assets/Ass ...         if (this.onerror) {
src/assets/Ass ...           this.onerror();
src/assets/Ass ...         }
src/assets/Ass ...       }
src/assets/Ass ...     };
src/assets/Ass ...   }
src/assets/Ass ...
src/assets/Ass ...   getImage(key) {
src/assets/Ass ...     return this.status[key] === 'loaded' ? this.images[key] : null;
src/assets/Ass ...   }
src/assets/Ass ... }
src/assets/Ass ...
src/assets/Ass ... module.exports = AssetManager;
src/assets/Ass ...
src/audio/Soun ... // ===== 文件：src/audio/SoundManager.js =====
src/audio/Soun ... 'use strict';
src/audio/Soun ...
src/audio/Soun ... const { THEME_IDS, THEMES } = require('../levels/Themes');
src/audio/Soun ...
src/audio/Soun ... const BGM_TRACKS = {
src/audio/Soun ...   [THEME_IDS.CASTLE]: THEMES[THEME_IDS.CASTLE].music,
src/audio/Soun ...   [THEME_IDS.OCEAN]: THEMES[THEME_IDS.OCEAN].music,
src/audio/Soun ...   [THEME_IDS.PENGUIN_HOTEL]: THEMES[THEME_IDS.PENGUIN_HOTEL].music,
src/audio/Soun ...   [THEME_IDS.DINOSAUR_PARK]: THEMES[THEME_IDS.DINOSAUR_PARK].music,
src/audio/Soun ...   [THEME_IDS.SKY_CITY]: THEMES[THEME_IDS.SKY_CITY].music,
src/audio/Soun ...   [THEME_IDS.SEA_TRAIN]: THEMES[THEME_IDS.SEA_TRAIN].music
src/audio/Soun ... };
src/audio/Soun ...
```

```
src/audio/Soun ... const EFFECT_TRACKS = {
src/audio/Soun ...   vanish: 'assets/audio/sfx-vanish.wav'
src/audio/Soun ... };
src/audio/Soun ...
src/audio/Soun ... class SoundManager {
src/audio/Soun ...   constructor(wxApi) {
src/audio/Soun ...     this.wx = wxApi;
src/audio/Soun ...     this.soundEnabled = true;
src/audio/Soun ...     this.currentTrack = null;
src/audio/Soun ...     this.music = null;
src/audio/Soun ...     this.effect = null;
src/audio/Soun ...
src/audio/Soun ...     if (wxApi && wxApi.createInnerAudioContext) {
src/audio/Soun ...       this.music = wxApi.createInnerAudioContext();
src/audio/Soun ...       this.music.loop = true;
src/audio/Soun ...       this.music.volume = 0.38;
src/audio/Soun ...       this.effect = wxApi.createInnerAudioContext();
src/audio/Soun ...       this.effect.loop = false;
src/audio/Soun ...       this.effect.volume = 0.72;
src/audio/Soun ...     }
src/audio/Soun ...   }
src/audio/Soun ...
src/audio/Soun ...   play(eventName) {
src/audio/Soun ...     const track = EFFECT_TRACKS[eventName];
src/audio/Soun ...     if (!track || !this.effect || !this.soundEnabled) {
src/audio/Soun ...       return;
src/audio/Soun ...     }
src/audio/Soun ...
src/audio/Soun ...     this.effect.stop();
src/audio/Soun ...     this.effect.src = track;
src/audio/Soun ...     this.effect.play();
src/audio/Soun ...   }
src/audio/Soun ...
src/audio/Soun ...   setSoundEnabled(enabled) {
src/audio/Soun ...     this.soundEnabled = enabled !== false;
src/audio/Soun ...     if (!this.soundEnabled && this.effect) {
src/audio/Soun ...       this.effect.stop();
src/audio/Soun ...     }
src/audio/Soun ...     if (!this.music) {
src/audio/Soun ...       return;
src/audio/Soun ...     }
src/audio/Soun ...
src/audio/Soun ...     if (!this.soundEnabled) {
src/audio/Soun ...       this.music.pause();
src/audio/Soun ...     } else if (this.currentTrack) {
src/audio/Soun ...       this.music.play();
src/audio/Soun ...     }
src/audio/Soun ...   }
src/audio/Soun ...
src/audio/Soun ...   playMusic(themeId) {
```

```
src/audio/Soun ...     const track = BGM_TRACKS[themeId];
src/audio/Soun ...     if (!track || !this.music) {
src/audio/Soun ...       return;
src/audio/Soun ...     }
src/audio/Soun ...
src/audio/Soun ...     const changedTrack = track !== this.currentTrack;
src/audio/Soun ...     if (changedTrack) {
src/audio/Soun ...       this.currentTrack = track;
src/audio/Soun ...       this.music.src = track;
src/audio/Soun ...     }
src/audio/Soun ...
src/audio/Soun ...     if (this.soundEnabled && changedTrack) {
src/audio/Soun ...       this.music.play();
src/audio/Soun ...     }
src/audio/Soun ...   }
src/audio/Soun ...
src/audio/Soun ...   stopMusic() {
src/audio/Soun ...     if (this.music) {
src/audio/Soun ...       this.music.stop();
src/audio/Soun ...     }
src/audio/Soun ...     this.currentTrack = null;
src/audio/Soun ...   }
src/audio/Soun ...
src/audio/Soun ...   vibrateDamage() {
src/audio/Soun ...     if (this.wx && this.wx.vibrateShort) {
src/audio/Soun ...       this.wx.vibrateShort();
src/audio/Soun ...     }
src/audio/Soun ...   }
src/audio/Soun ... }
src/audio/Soun ...
src/audio/Soun ... SoundManager.BGM_TRACKS = BGM_TRACKS;
src/audio/Soun ... SoundManager.EFFECT_TRACKS = EFFECT_TRACKS;
src/audio/Soun ...
src/audio/Soun ... module.exports = SoundManager;
src/audio/Soun ...
 src/ui/AudioTo ... // ===== 文件：src/ui/AudioToggle.js =====
 src/ui/AudioTo ... 'use strict';
 src/ui/AudioTo ...
 src/ui/AudioTo ... function getAudioToggleBounds(width) {
 src/ui/AudioTo ...   return {
 src/ui/AudioTo ...     x: width - 91,
 src/ui/AudioTo ...     y: 128,
 src/ui/AudioTo ...     width: 75,
 src/ui/AudioTo ...     height: 34
 src/ui/AudioTo ...   };
 src/ui/AudioTo ... }
 src/ui/AudioTo ...
 src/ui/AudioTo ... function isAudioToggleHit(width, point) {
 src/ui/AudioTo ...   if (!point) {
 src/ui/AudioTo ...     return false;
```

```
src/ui/AudioTo ...   }
src/ui/AudioTo ...
src/ui/AudioTo ...   const button = getAudioToggleBounds(width);
src/ui/AudioTo ...   return (
src/ui/AudioTo ...     point.x >= button.x &&
src/ui/AudioTo ...     point.x <= button.x + button.width &&
src/ui/AudioTo ...     point.y >= button.y &&
src/ui/AudioTo ...     point.y <= button.y + button.height
src/ui/AudioTo ...   );
src/ui/AudioTo ... }
src/ui/AudioTo ...
src/ui/AudioTo ... module.exports = {
src/ui/AudioTo ...   getAudioToggleBounds,
src/ui/AudioTo ...   isAudioToggleHit
src/ui/AudioTo ... };
src/ui/AudioTo ...
src/ui/Difficulty ... // ===== 文件：src/ui/DifficultySelector.js =====
src/ui/Difficulty ... 'use strict';
src/ui/Difficulty ...
src/ui/Difficulty ... const DIFFICULTY_MODES = {
src/ui/Difficulty ...   NORMAL: 'normal',
src/ui/Difficulty ...   PLUS_ONE: 'plus-one'
src/ui/Difficulty ... };
src/ui/Difficulty ...
src/ui/Difficulty ... const DIFFICULTIES = {
src/ui/Difficulty ...   normal: {
src/ui/Difficulty ...     id: DIFFICULTY_MODES.NORMAL,
src/ui/Difficulty ...     label: '普通难度',
src/ui/Difficulty ...     detail: '标准速度',
src/ui/Difficulty ...     timeScale: 1
src/ui/Difficulty ...   },
src/ui/Difficulty ...   'plus-one': {
src/ui/Difficulty ...     id: DIFFICULTY_MODES.PLUS_ONE,
src/ui/Difficulty ...     label: '+1 难度',
src/ui/Difficulty ...     detail: '速度 +50%',
src/ui/Difficulty ...     timeScale: 1.5
src/ui/Difficulty ...   }
src/ui/Difficulty ... };
src/ui/Difficulty ...
src/ui/Difficulty ... function getDifficulty(mode) {
src/ui/Difficulty ...   return DIFFICULTIES[mode] || null;
src/ui/Difficulty ... }
src/ui/Difficulty ...
src/ui/Difficulty ... function getDifficultyButtons(width, height) {
src/ui/Difficulty ...   const buttonWidth = Math.min(288, width - 48);
src/ui/Difficulty ...   const buttonHeight = 58;
src/ui/Difficulty ...   const x = (width - buttonWidth) * 0.5;
src/ui/Difficulty ...   const startY = height * 0.53;
src/ui/Difficulty ...   const gap = 14;
src/ui/Difficulty ...
```

```
src/ui/Difficulty ...   return [DIFFICULTY_MODES.NORMAL, DIFFICULTY_MODES.PLUS_ONE].map((mode, index) => {
src/ui/Difficulty ...     const difficulty = getDifficulty(mode);
src/ui/Difficulty ...     return {
src/ui/Difficulty ...       mode,
src/ui/Difficulty ...       label: difficulty.label,
src/ui/Difficulty ...       detail: difficulty.detail,
src/ui/Difficulty ...       x,
src/ui/Difficulty ...       y: startY + index * (buttonHeight + gap),
src/ui/Difficulty ...       width: buttonWidth,
src/ui/Difficulty ...       height: buttonHeight
src/ui/Difficulty ...     };
src/ui/Difficulty ...   });
src/ui/Difficulty ... }
src/ui/Difficulty ...
src/ui/Difficulty ... function findDifficultyAtPoint(width, height, point) {
src/ui/Difficulty ...   if (!point) {
src/ui/Difficulty ...     return null;
src/ui/Difficulty ...   }
src/ui/Difficulty ...
src/ui/Difficulty ...   const buttons = getDifficultyButtons(width, height);
src/ui/Difficulty ...   for (let i = 0; i < buttons.length; i += 1) {
src/ui/Difficulty ...     const button = buttons[i];
src/ui/Difficulty ...     if (
src/ui/Difficulty ...       point.x >= button.x &&
src/ui/Difficulty ...       point.x <= button.x + button.width &&
src/ui/Difficulty ...       point.y >= button.y &&
src/ui/Difficulty ...       point.y <= button.y + button.height
src/ui/Difficulty ...     ) {
src/ui/Difficulty ...       return button.mode;
src/ui/Difficulty ...     }
src/ui/Difficulty ...   }
src/ui/Difficulty ...
src/ui/Difficulty ...   return null;
src/ui/Difficulty ... }
src/ui/Difficulty ...
src/ui/Difficulty ... module.exports = {
src/ui/Difficulty ...   DIFFICULTY_MODES,
src/ui/Difficulty ...   getDifficulty,
src/ui/Difficulty ...   getDifficultyButtons,
src/ui/Difficulty ...   findDifficultyAtPoint
src/ui/Difficulty ... };
src/ui/Difficulty ...
 src/ui/ItemBar.js // ===== 文件：src/ui/ItemBar.js =====
src/ui/ItemBar.j ... 'use strict';
src/ui/ItemBar.j ...
src/ui/ItemBar.j ... const ITEM_TYPES = {
src/ui/ItemBar.j ...   HEALTH_POTION: 'healthPotion',
src/ui/ItemBar.j ...   COMBO_CHAIN: 'comboChain'
src/ui/ItemBar.j ... };
src/ui/ItemBar.j ...
```

```
src/ui/ItemBar.j ... const ITEM_ORDER = [
src/ui/ItemBar.j ...   ITEM_TYPES.HEALTH_POTION,
src/ui/ItemBar.j ...   ITEM_TYPES.COMBO_CHAIN
src/ui/ItemBar.j ... ];
src/ui/ItemBar.j ...
src/ui/ItemBar.j ... const ITEM_SLOT_SIZE = 56;
src/ui/ItemBar.j ... const ITEM_SLOT_GAP = 10;
src/ui/ItemBar.j ... const ITEM_SLOT_LEFT = 14;
src/ui/ItemBar.j ... const HEART_SLOT_LEFT = 28;
src/ui/ItemBar.j ... const HEART_SLOT_GAP = 26;
src/ui/ItemBar.j ... const HEART_SLOT_BOTTOM = 28;
src/ui/ItemBar.j ...
src/ui/ItemBar.j ... function getHeartSlotPosition(width, height, heartIndex) {
src/ui/ItemBar.j ...   return {
src/ui/ItemBar.j ...     x: HEART_SLOT_LEFT + heartIndex * HEART_SLOT_GAP,
src/ui/ItemBar.j ...     y: height - HEART_SLOT_BOTTOM
src/ui/ItemBar.j ...   };
src/ui/ItemBar.j ... }
src/ui/ItemBar.j ...
src/ui/ItemBar.j ... function getItemSlots(width, height, items) {
src/ui/ItemBar.j ...   const inventory = items || {};
src/ui/ItemBar.j ...   const visibleItems = ITEM_ORDER.filter((type) => (inventory[type] || 0) > 0);
src/ui/ItemBar.j ...   const barHeight = visibleItems.length * ITEM_SLOT_SIZE + Math.max(0, visibleItems.length - 1) * ITEM_SLOT_GAP;
src/ui/ItemBar.j ...   const bottomSlotY = (height + barHeight) * 0.5 - ITEM_SLOT_SIZE;
src/ui/ItemBar.j ...   const slots = [];
src/ui/ItemBar.j ...
src/ui/ItemBar.j ...   for (let i = 0; i < visibleItems.length; i += 1) {
src/ui/ItemBar.j ...     const type = visibleItems[i];
src/ui/ItemBar.j ...     const count = inventory[type] || 0;
src/ui/ItemBar.j ...
src/ui/ItemBar.j ...     slots.push({
src/ui/ItemBar.j ...       type,
src/ui/ItemBar.j ...       count,
src/ui/ItemBar.j ...       x: ITEM_SLOT_LEFT,
src/ui/ItemBar.j ...       y: bottomSlotY - slots.length * (ITEM_SLOT_SIZE + ITEM_SLOT_GAP),
src/ui/ItemBar.j ...       width: ITEM_SLOT_SIZE,
src/ui/ItemBar.j ...       height: ITEM_SLOT_SIZE
src/ui/ItemBar.j ...     });
src/ui/ItemBar.j ...   }
src/ui/ItemBar.j ...
src/ui/ItemBar.j ...   return slots;
src/ui/ItemBar.j ... }
src/ui/ItemBar.j ...
src/ui/ItemBar.j ... function findItemAtPoint(width, height, items, point) {
src/ui/ItemBar.j ...   if (!point) {
src/ui/ItemBar.j ...     return null;
src/ui/ItemBar.j ...   }
src/ui/ItemBar.j ...
src/ui/ItemBar.j ...   const slots = getItemSlots(width, height, items);
src/ui/ItemBar.j ...   for (let i = 0; i < slots.length; i += 1) {
```

```
src/ui/ItemBar.j ...     const slot = slots[i];
src/ui/ItemBar.j ...     if (
src/ui/ItemBar.j ...       point.x >= slot.x &&
src/ui/ItemBar.j ...       point.x <= slot.x + slot.width &&
src/ui/ItemBar.j ...       point.y >= slot.y &&
src/ui/ItemBar.j ...       point.y <= slot.y + slot.height
src/ui/ItemBar.j ...     ) {
src/ui/ItemBar.j ...       return slot.type;
src/ui/ItemBar.j ...     }
src/ui/ItemBar.j ...   }
src/ui/ItemBar.j ...
src/ui/ItemBar.j ...   return null;
src/ui/ItemBar.j ... }
src/ui/ItemBar.j ...
src/ui/ItemBar.j ... module.exports = {
src/ui/ItemBar.j ...   ITEM_TYPES,
src/ui/ItemBar.j ...   getHeartSlotPosition,
src/ui/ItemBar.j ...   getItemSlots,
src/ui/ItemBar.j ...   findItemAtPoint
src/ui/ItemBar.j ... };
  src/ui/ItemBar.js
  src/utils/math.js // ===== 文件：src/utils/math.js =====
src/utils/math.js:1 'use strict';
src/utils/math.js:2
src/utils/math.js:3 function clamp(value, min, max) {
src/utils/math.js:4   return Math.max(min, Math.min(max, value));
src/utils/math.js:5 }
src/utils/math.js:6
src/utils/math.js:7 function distance(a, b) {
src/utils/math.js:8   const dx = a.x - b.x;
src/utils/math.js:9   const dy = a.y - b.y;
 src/utils/math.j ...   return Math.sqrt(dx * dx + dy * dy);
 src/utils/math.j ... }
 src/utils/math.j ...
 src/utils/math.j ... function normalizeVector(dx, dy) {
 src/utils/math.j ...   const len = Math.sqrt(dx * dx + dy * dy) || 1;
 src/utils/math.j ...   return { x: dx / len, y: dy / len, len };
 src/utils/math.j ... }
 src/utils/math.j ...
 src/utils/math.j ... function lerp(a, b, t) {
 src/utils/math.j ...   return a + (b - a) * t;
 src/utils/math.j ... }
 src/utils/math.j ...
 src/utils/math.j ... module.exports = {
 src/utils/math.j ...   clamp,
 src/utils/math.j ...   distance,
 src/utils/math.j ...   normalizeVector,
 src/utils/math.j ...   lerp
 src/utils/math.j ... };
   src/utils/math.js
```